



**D'ANGELO ANTONIO  
MAFFEO MARCO  
RICCARDI STEFANO  
TORELLA GAETANO**

# DUCKIETOWN

**DIFFERENTIAL MOBILE ROBOT CONTROL**

CONTROL LAB – PROF. RAFFAELE IERVOLINO



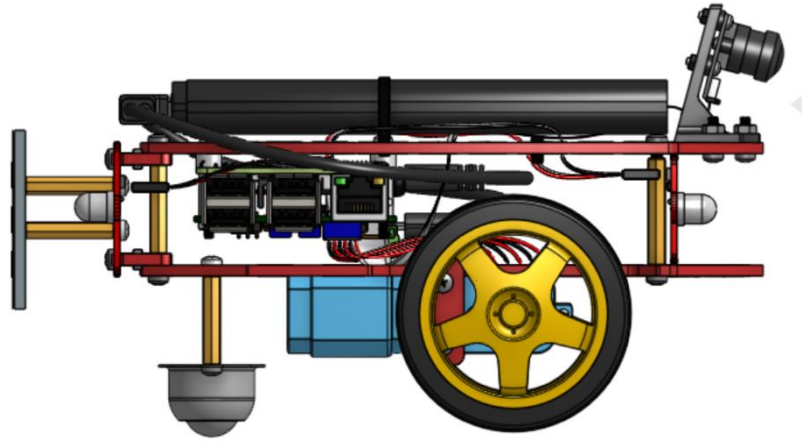
**MOLARO VINCENZO  
TUCCILLO DOMENICO  
VITONE CARLO  
VODOLA BENITO**

# SUMMARY

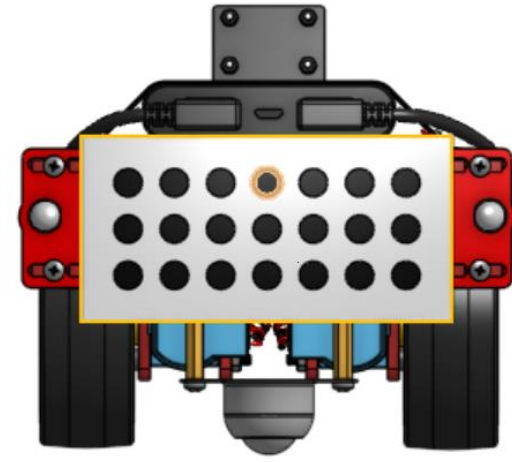
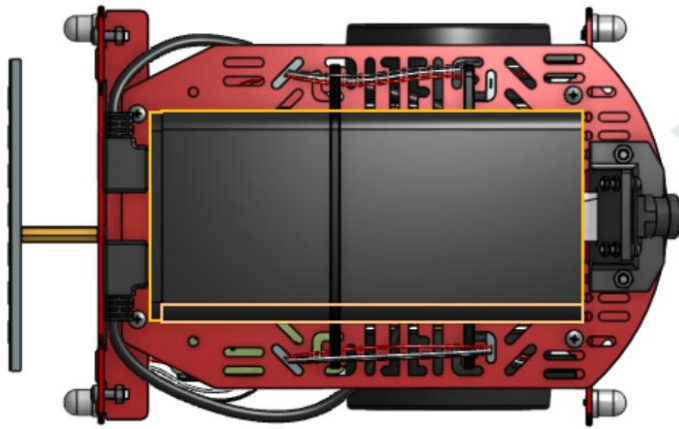
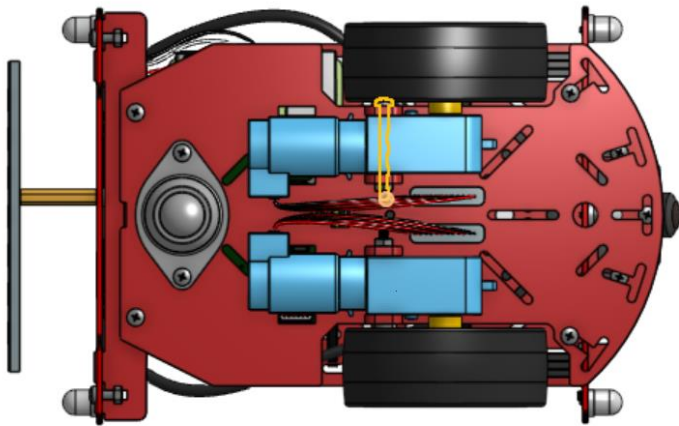
- **Assembly and set-up**
- **Communication**
- **Control**
- **Vision control**
- **Hands On**
- **Future implementation**



# DUCKIEBOT



The Duckiebot is a differential drive robot, designed and developed by Massachusetts Institute of Technology. The aim of Duckiebot project is not only to control the robot in order to control it, but to make it move appropriately in a town.



# BUILDING

Firstly we build the Duckiebot using the website guide on [duckietown.com](https://duckietown.com)



## Guides for getting started

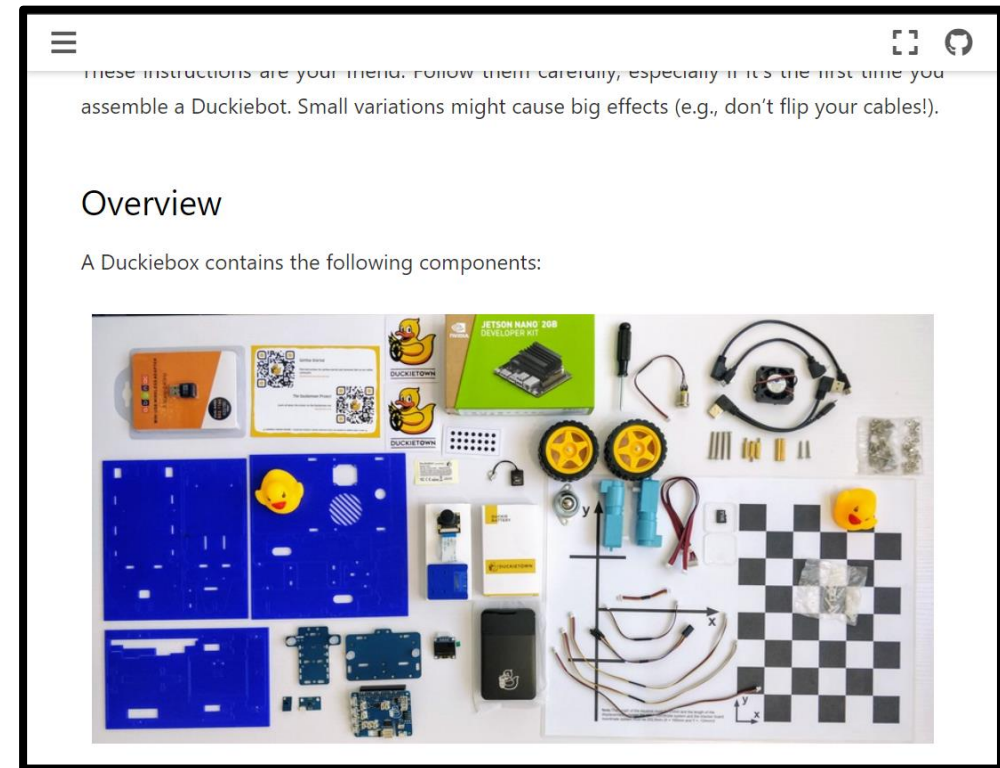
Duckietown is at the intersection of education, training, and research and is designed for instructors, researchers, and makademics.

I want to teach

I want to learn

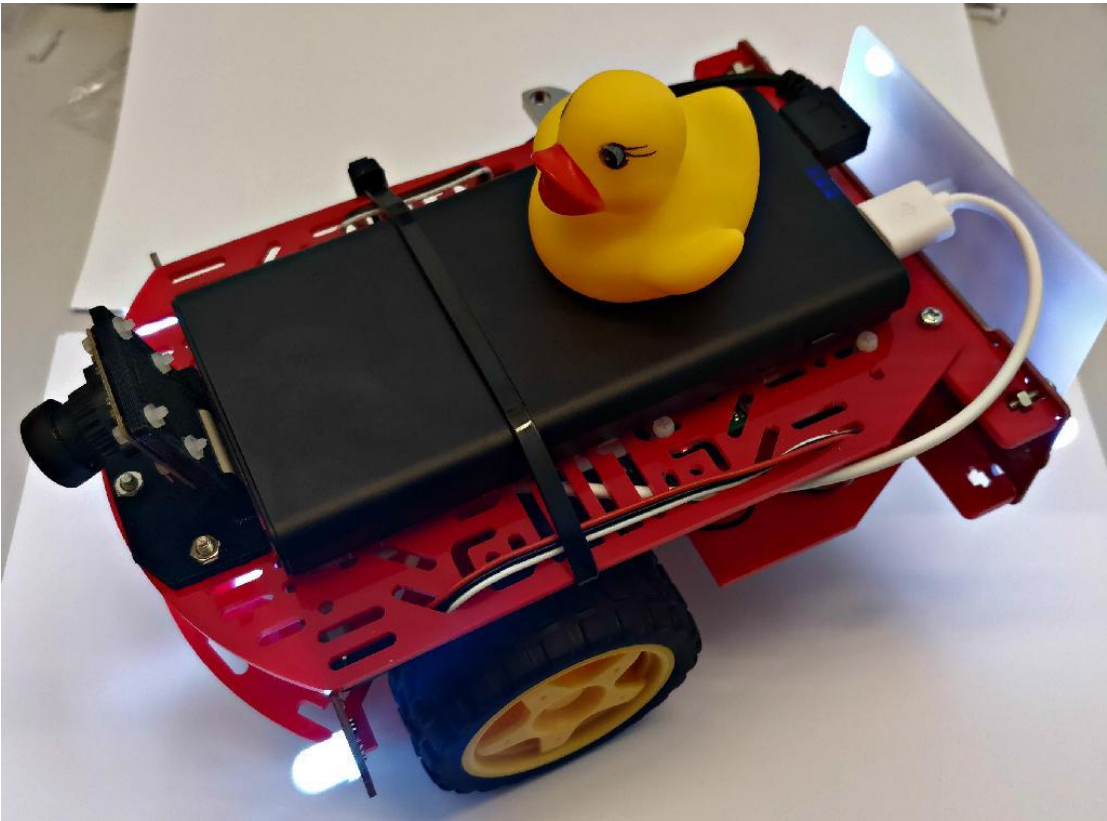
I want to research

I want to build



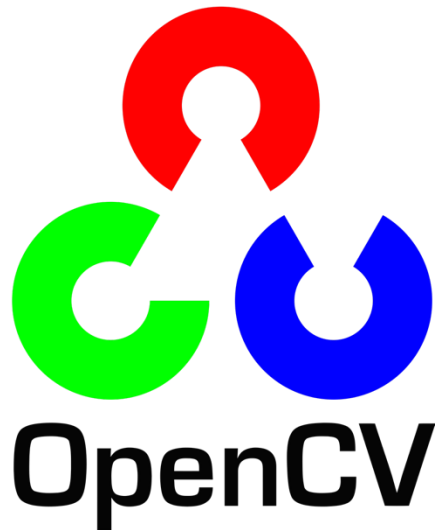


# BUILDING



# SETUP

ROS



# DB-V19 CAD MODEL

**TOP ASSEMBLY**

*Cam\_hold*

**SBC**

*Raspberry\_pi  
Duckiehat*

**WIRES**

**BATTERY**

**BACK**

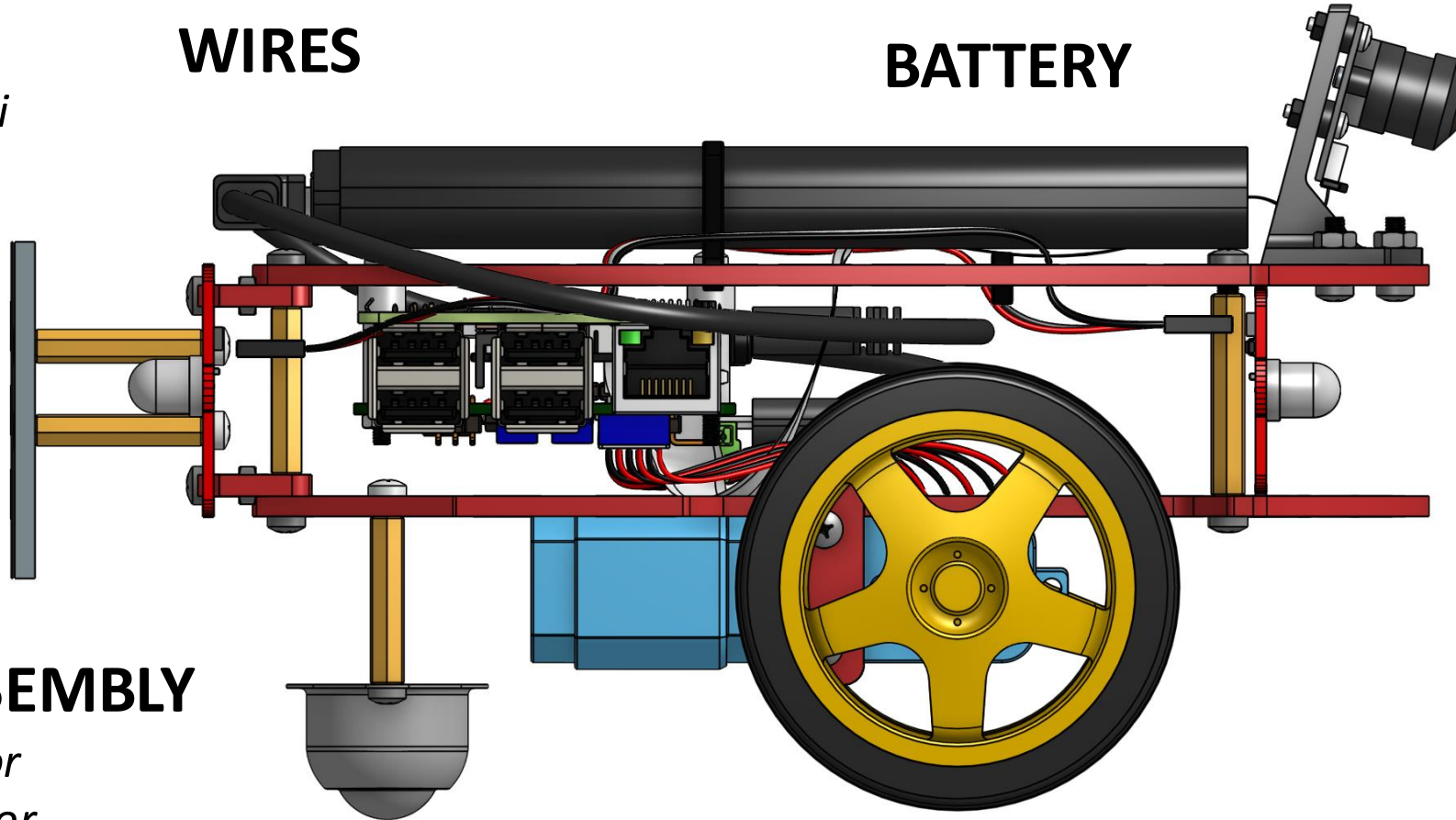
**ASSEMBLY**

*Backpattern\_plate*

**FRONT  
ASSEMBLY**

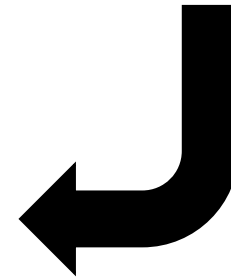
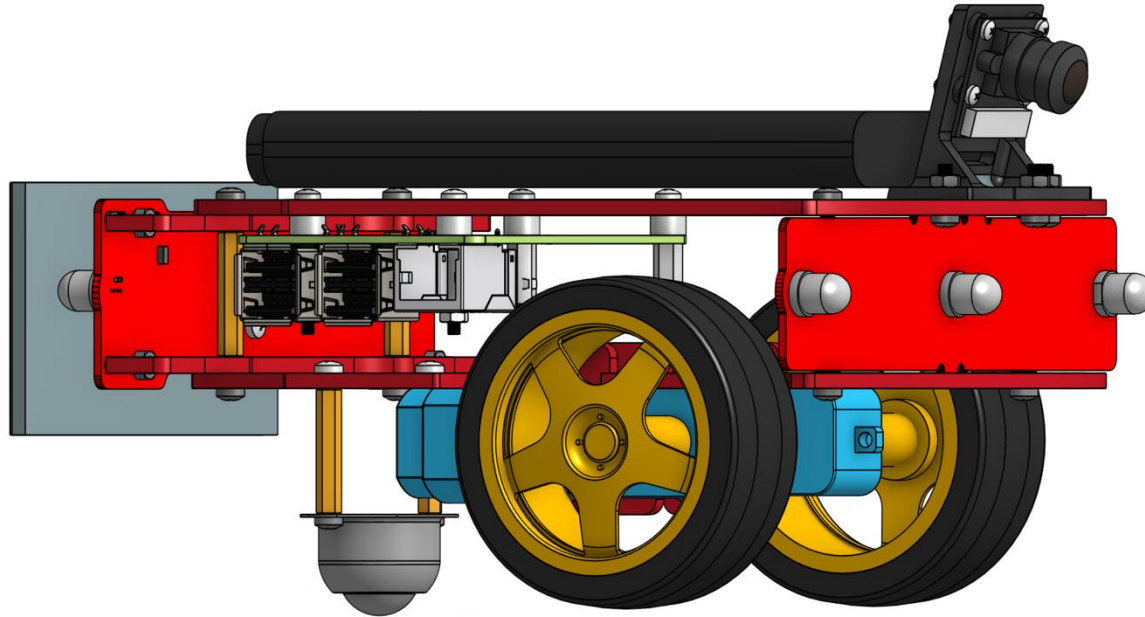
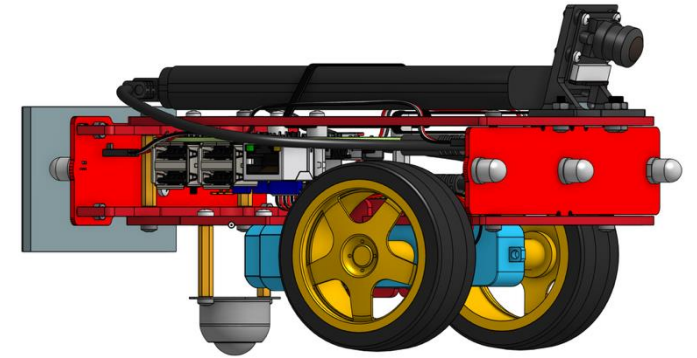
**BOTTOM ASSEMBLY**

*Left\_motor  
Right\_motor  
Hudson\_bearing*



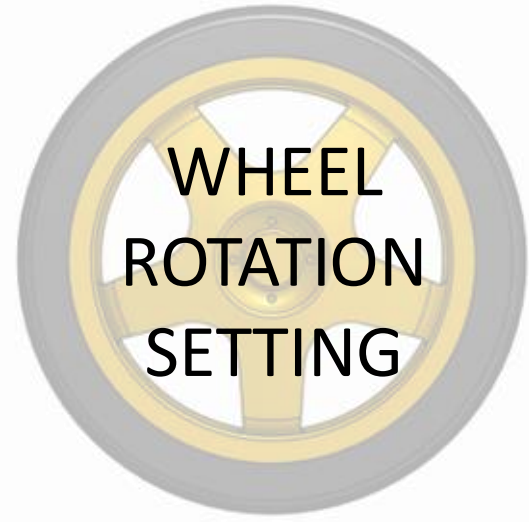
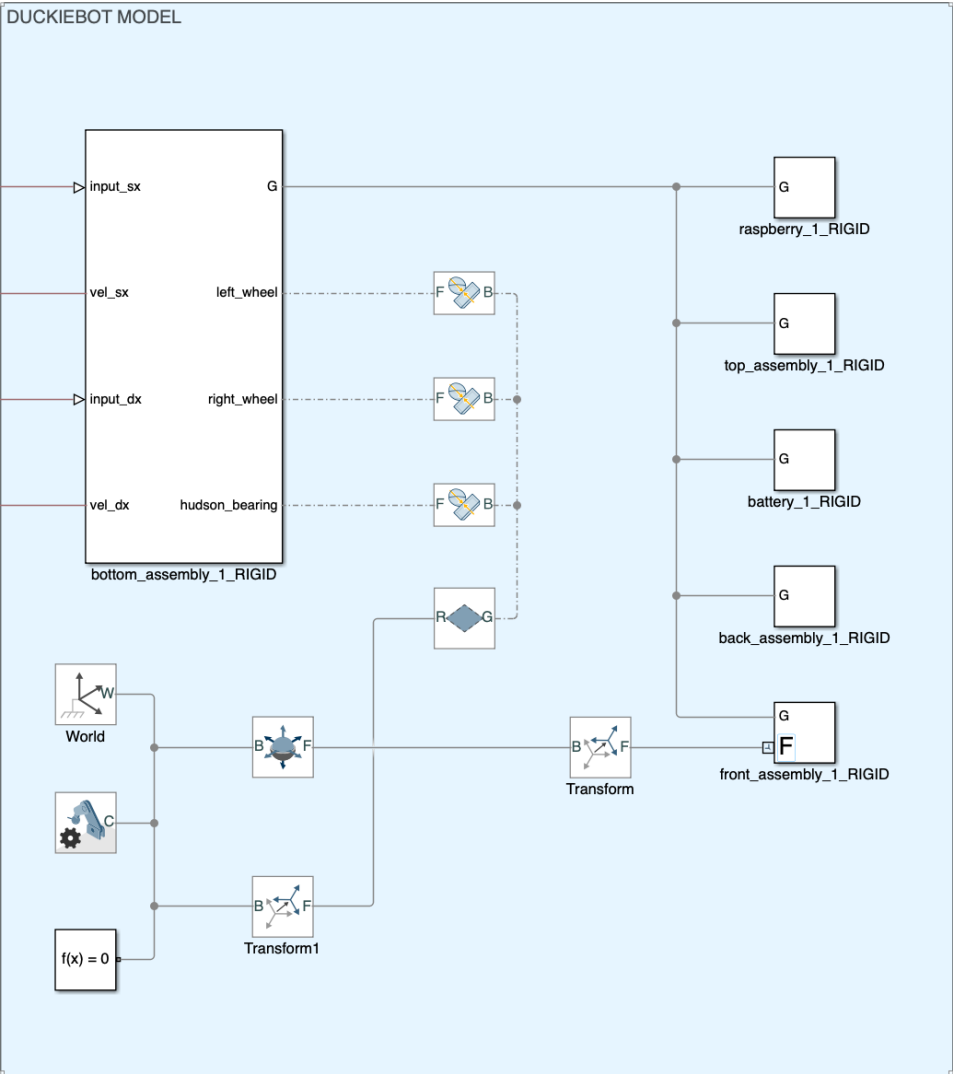


- Removed all those extra components such as screws, pins, bolts, cables, etc. that weighed down the model.
- Assign materials to components using Aluminum and PVC.
- Created a group that contained all the assemblies of the parts so that they could be joined with a fastener mating.



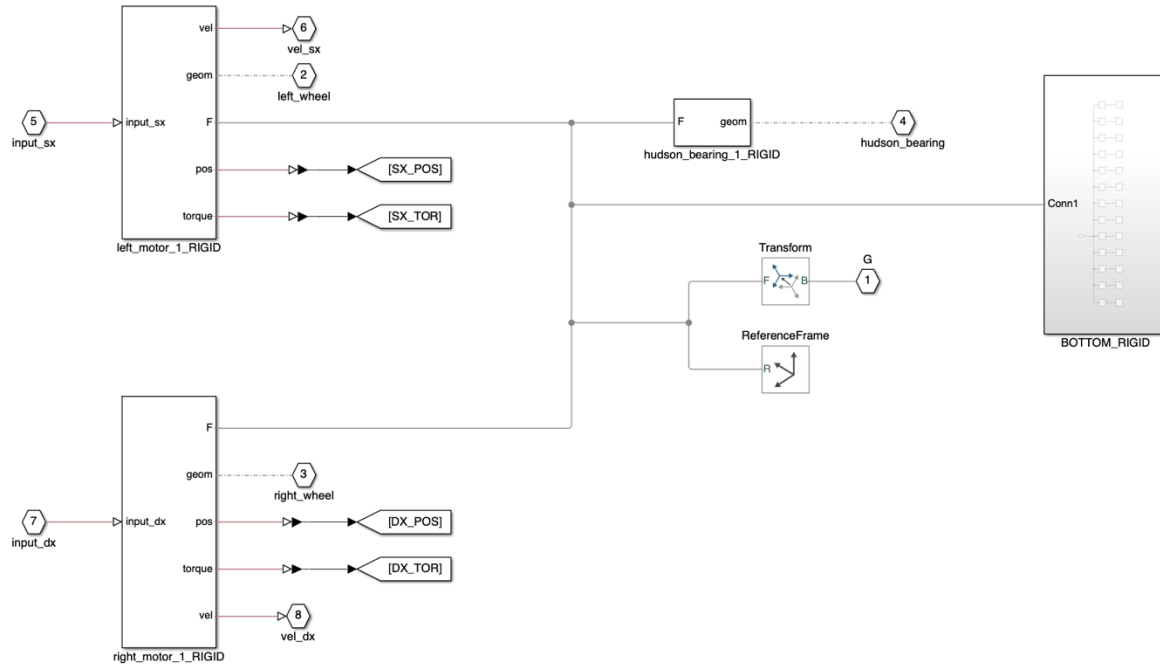


After importing the model from Onshape to Simulink we worked on it and get the following pattern.

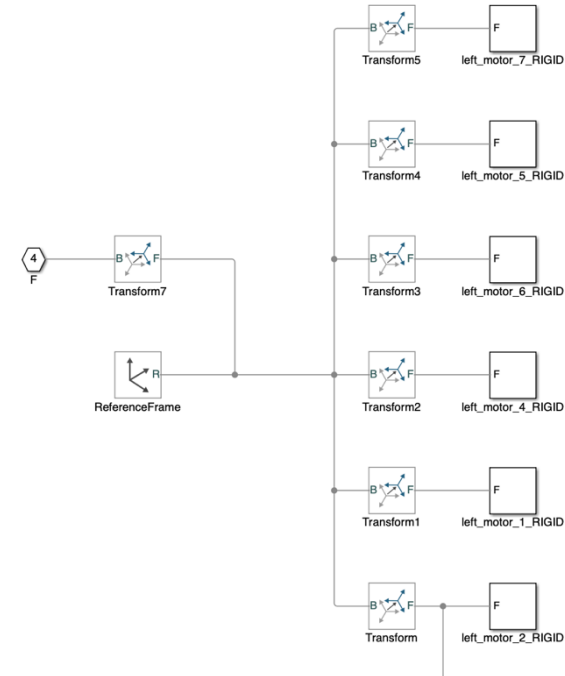


# WHEEL ROTATION SETTING

We focus on the bottom assembly, specifically inside two subsystems called `left_motor_1_RIGID` and `right_motor_1_RIGID` that contain the motors and wheels parts.

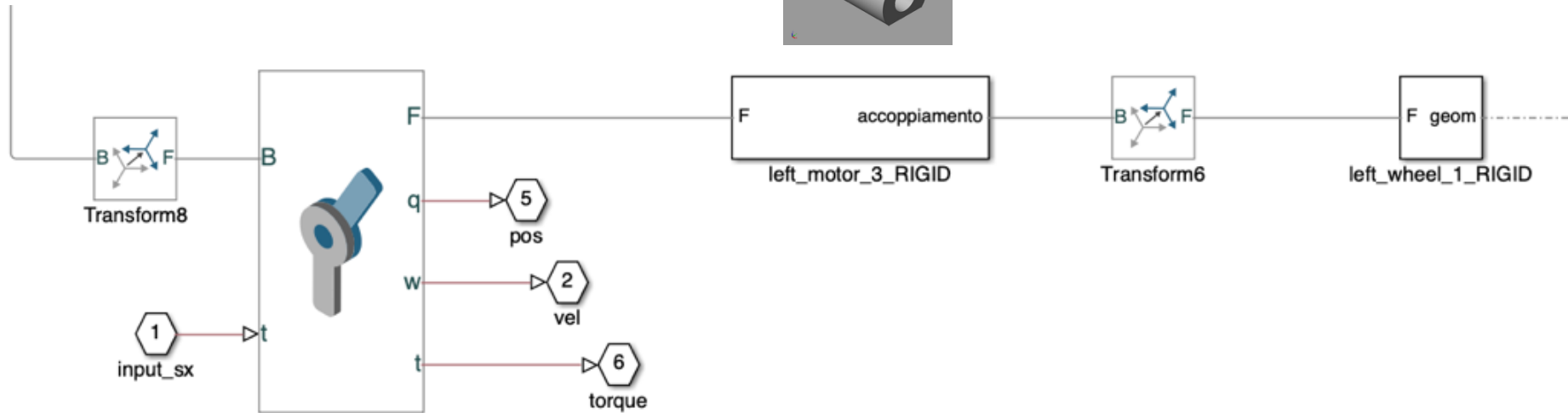
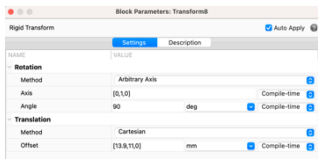
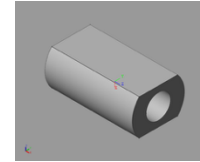
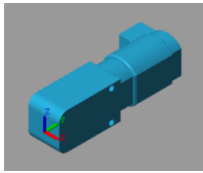


`bottom_assembly_1_RIGID`



`bottom_assembly_1_RIGID >> left_motor_1_RIGID`





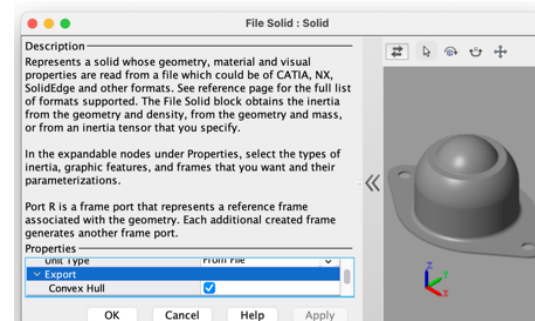
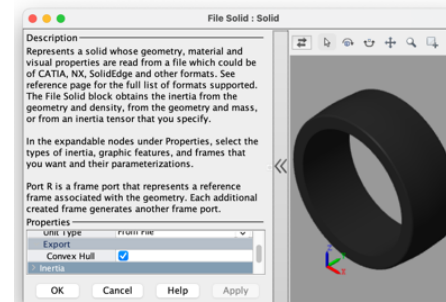
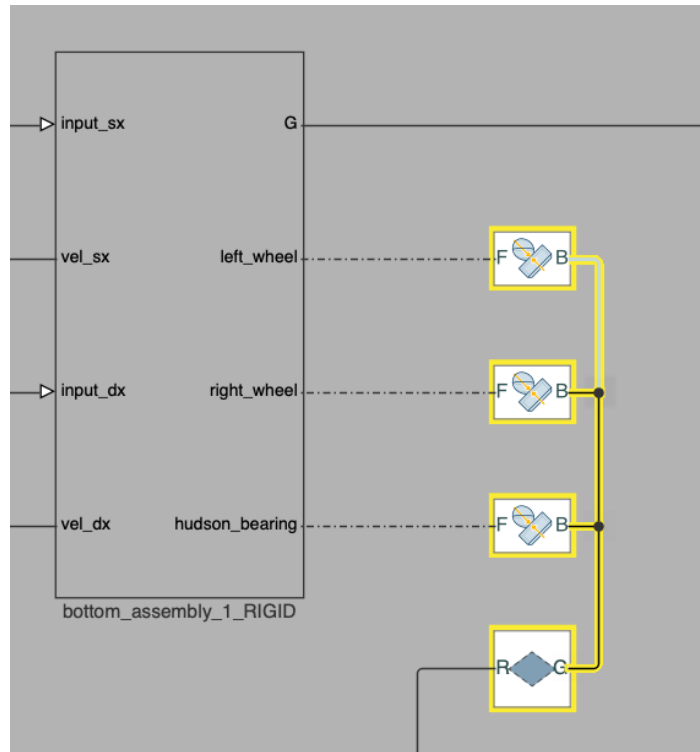
*bottom\_assembly\_1\_RIGID >> left\_motor\_1\_RIGID*

The rotation of the motor is simulated by the gray pin via a revolute joint driven by a torque input placed between the motor and the gray pin that will allow the pin to rotate with respect to an appropriately chosen reference frame by changing the values of the rotation and translation matrix.



# ROAD CONTACT

We created a plane of size 1x1 m to place the Duckiebot model. The idea is to place on this plane the 3 sliding elements which would be the wheels rubber and the Hudson bearing. To do this we used the Convex Hull function inside the solid part of each elements and finally connected the above elements to the plane using a contact constraint between this elements and the plane.

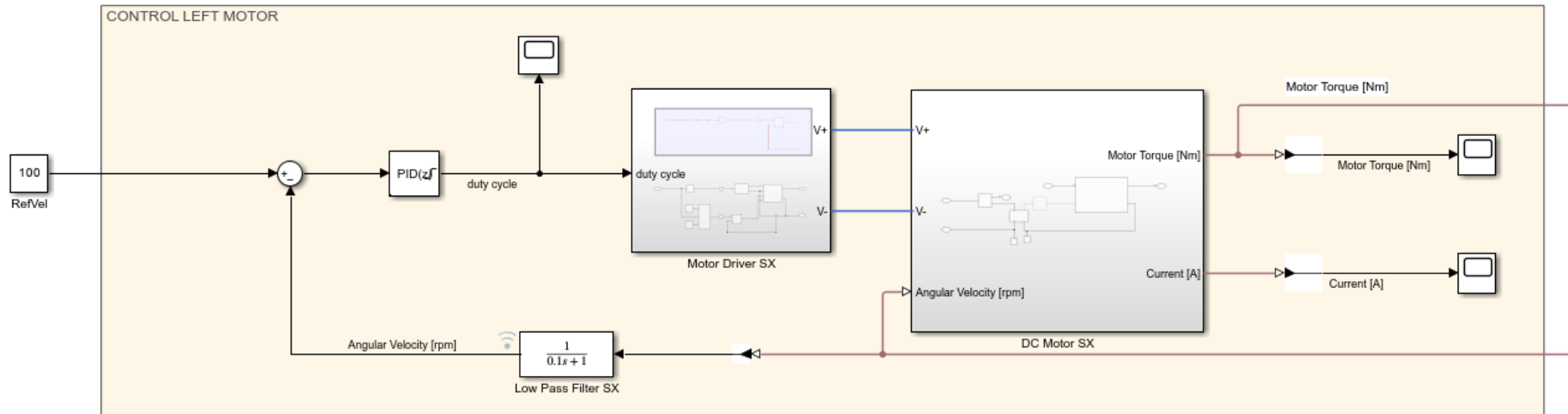




# MODEL CONTROL

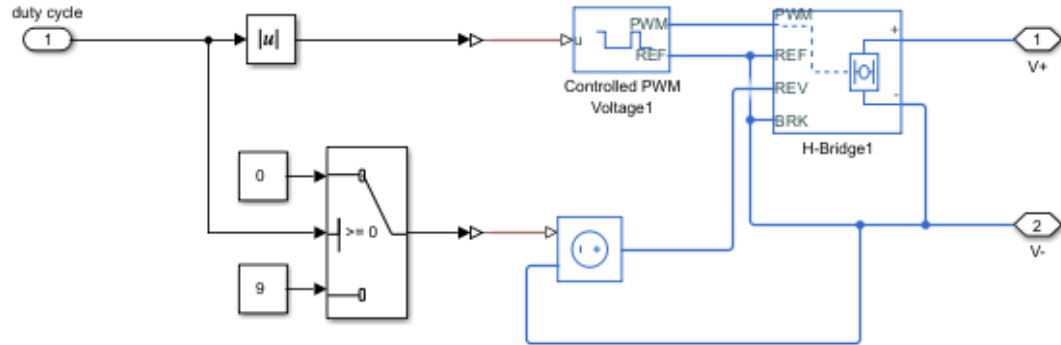
An early step to control our duckiebot was to implement a controller in a Simulink model created using SimScape. In this model we connect the revolute joint with a DC motor that have similar specifications respect to our motors.

The block Motor Driver SX is the one that control the left dc motor: it use the signal given by PID and use a PWM and an H-bridge to generate the voltage for the motor.



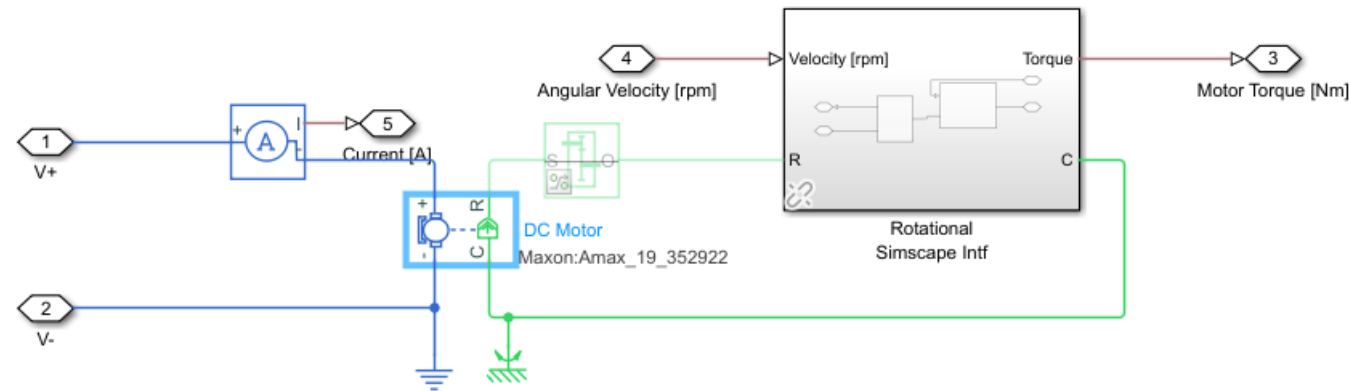
# DC MOTOR

## Motor Driver



- Duty cycle
- Abs
- Switch
- Controlled PWM Voltage
- H-Bridge

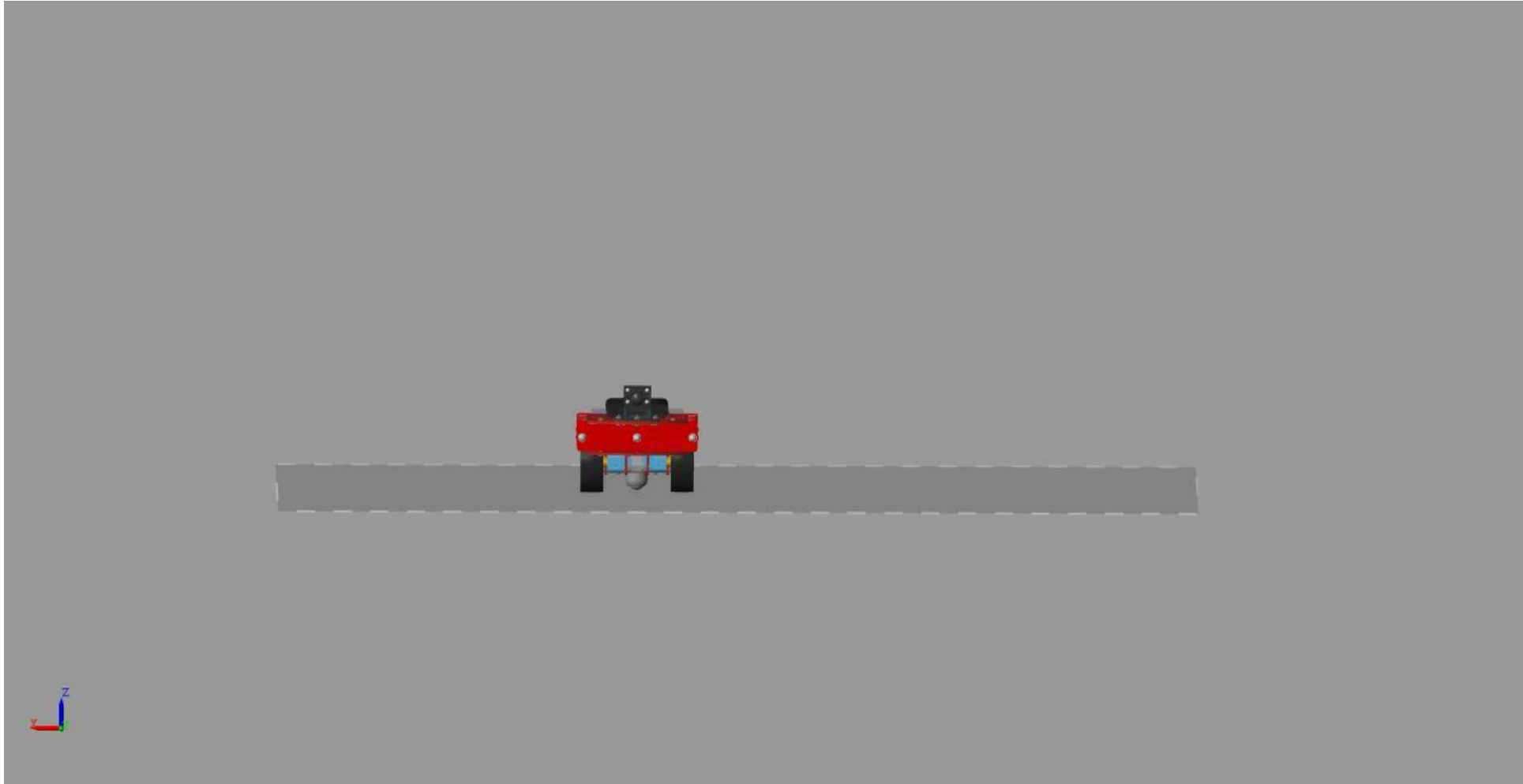
## DC Motor



- DC motor
- Rotational Simscape Interface

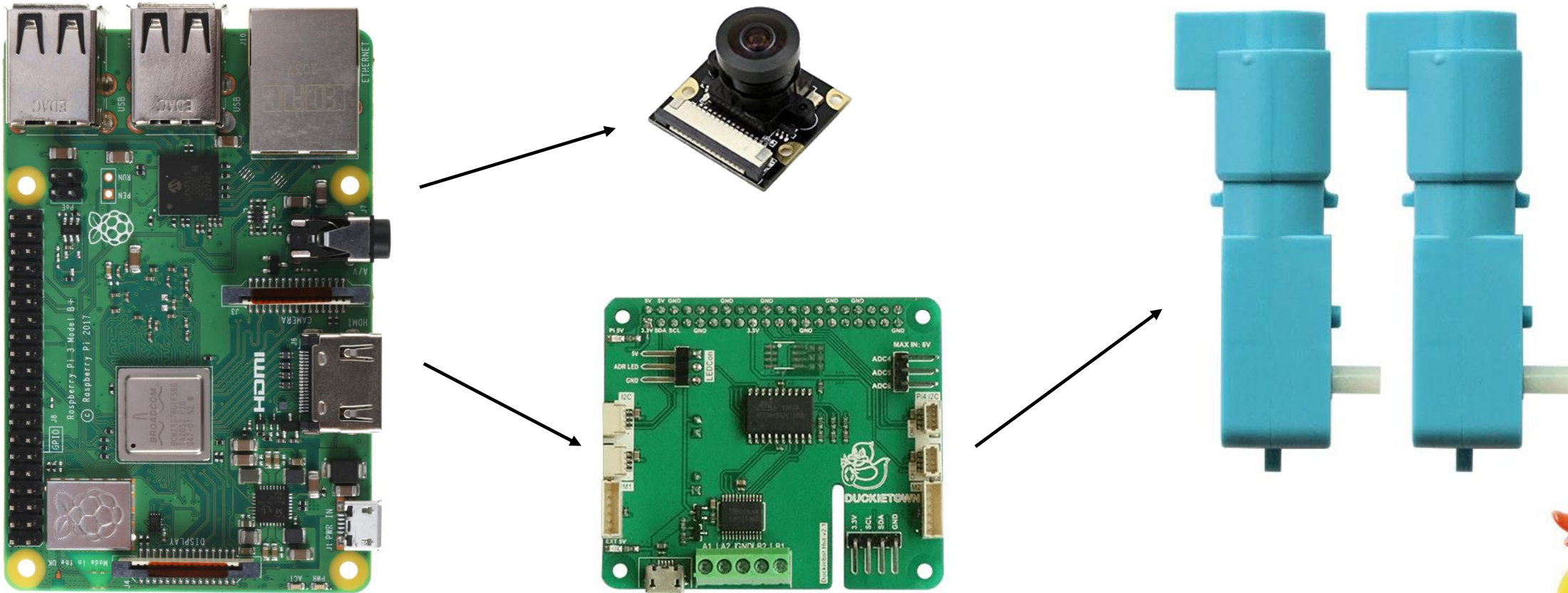


# 3D SIMULATION



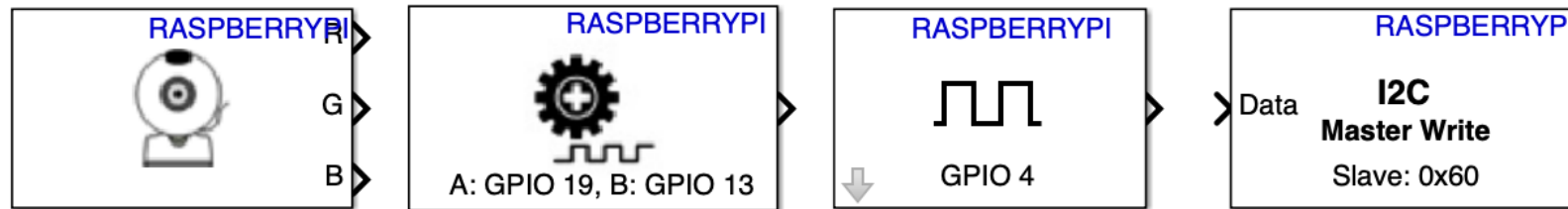
# RASPBERRY PI 3B+

The DuckieBot is equipped with a Raspberry 3B+ board which is connected to the camera and the HAT that controls the LEDs and motors



# FIRST CONTACT

During the first tests, we tried to derive the information coming from the motor encoders and the camera through the tools offered by the "Support Package for Raspberry Pi Hardware", the same thing was done to send commands to the engines:

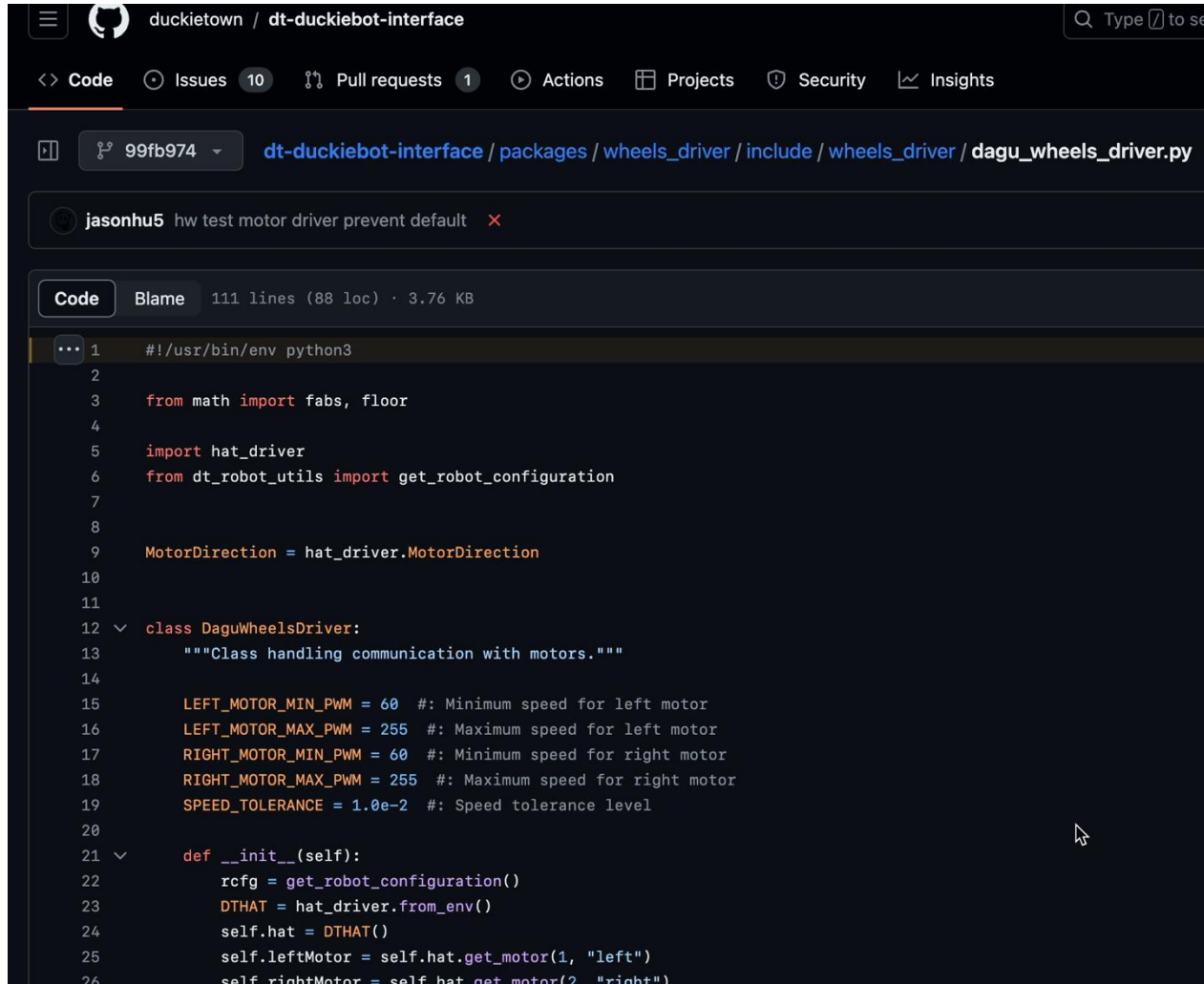


But we found that the Raspberry communicates with the engines through an I2C communication protocol conveniently encoded by the operating system installed by Duckietown.





# GITHUB & PYTHON

A screenshot of a GitHub web interface. The repository is 'duckietown / dt-duckiebot-interface'. The file path is 'packages / wheels\_driver / include / wheels\_driver / dagu\_wheels\_driver.py'. The file is 111 lines long, 88 loc, and 3.76 KB. The code is a Python script for a motor driver. It starts with a shebang line, imports 'math' for 'fabs' and 'floor', and imports 'hat\_driver' and 'get\_robot\_configuration' from 'dt\_robot\_utils'. It defines 'MotorDirection' as 'hat\_driver.MotorDirection'. Then it defines a class 'DaguWheelsDriver' with a docstring 'Class handling communication with motors.' and several constants for motor speeds and tolerance. Finally, it has an '\_\_init\_\_' method that initializes 'rcfg', 'DTHAT', 'self.hat', 'self.leftMotor', and 'self.rightMotor'.

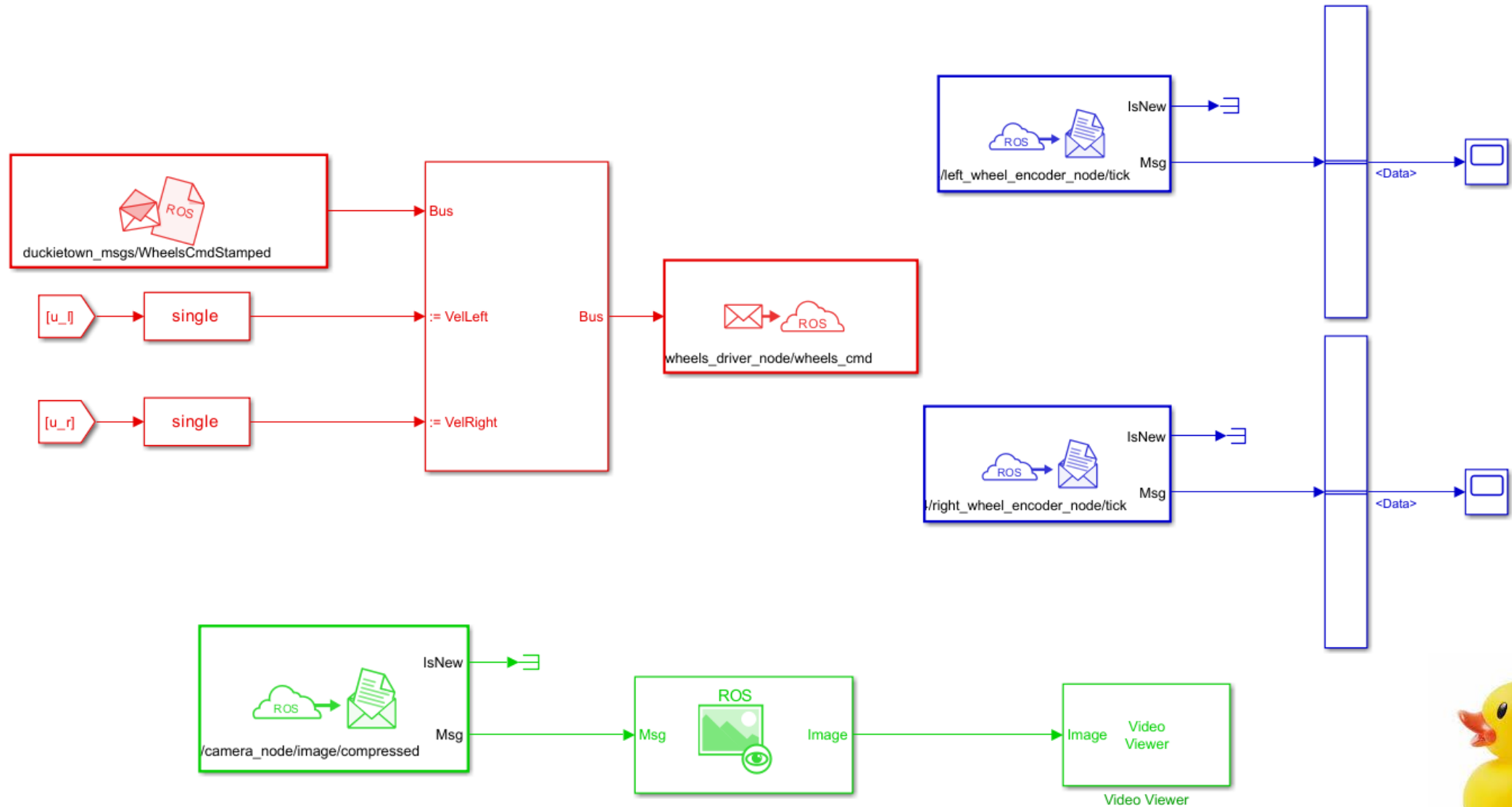
We have therefore focused our efforts on decoding the Python codes installed on the DuckieBot used for communication between the various components.

After days of reading thousands of lines of code and hundreds of files, we finally got...

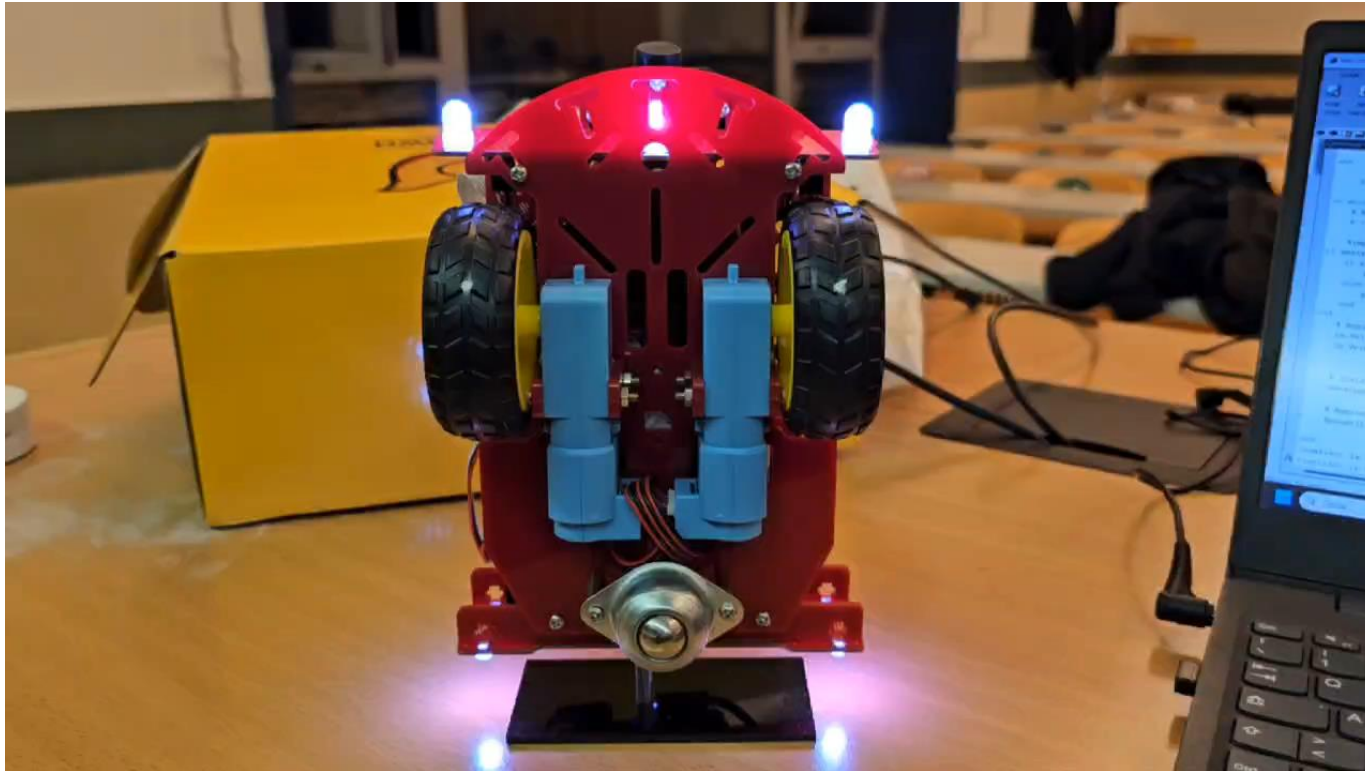
**NOTHING!**



# ROS BASED INTERFACE



# A SMALL STEP FOR THE DUCK...



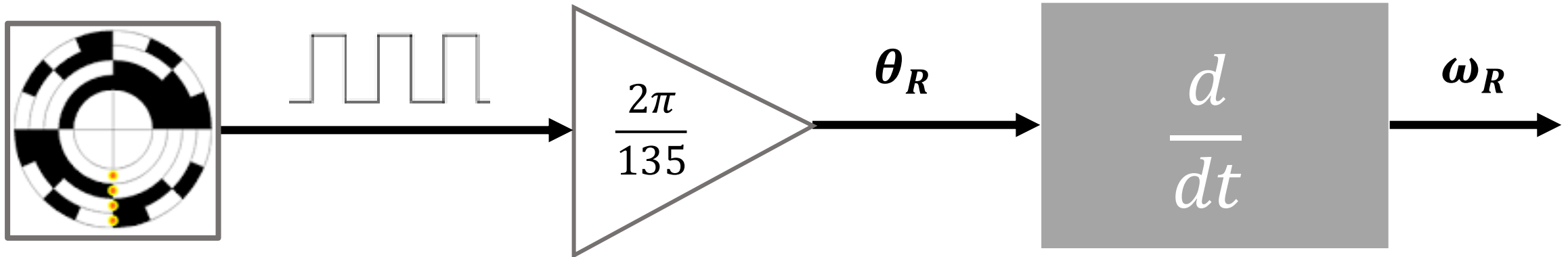
**Our first step was to actuate the wheels with a simple feedforward action, directly actuating the motor drives through the specific topic.**

**However, the feedforward action is not enough, since the wheels rotate with different speeds.**

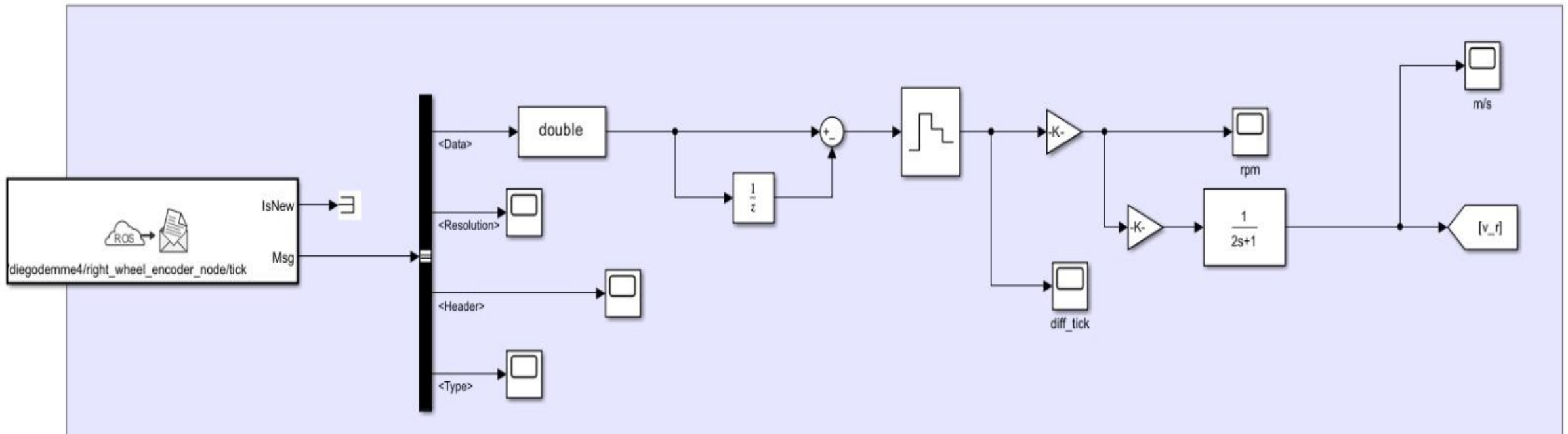


# VELOCITY RECONSTRUCTION

We can compute velocity thanks to a position measurement

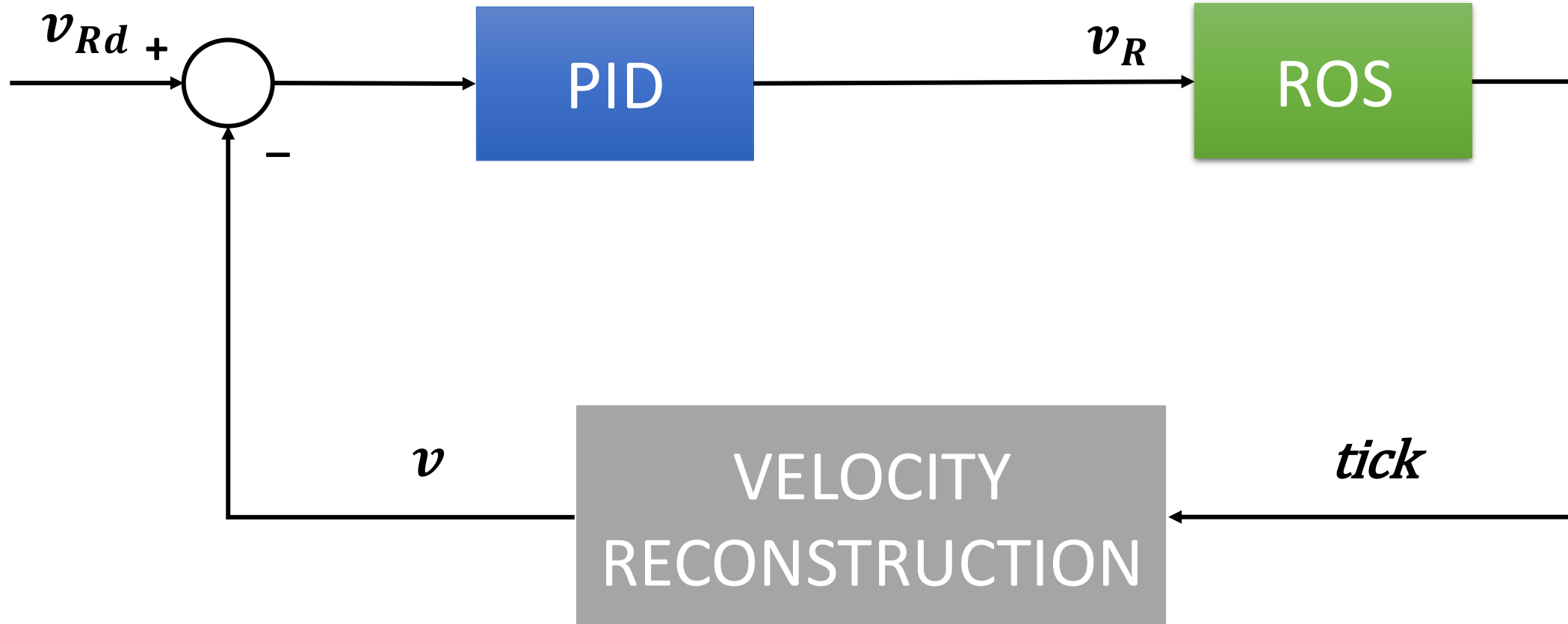


# VELOCITY RECONSTRUCTION



# INTERNAL LOOP

**We've designed two internal loops, one for each wheel, to precisely control the speed**

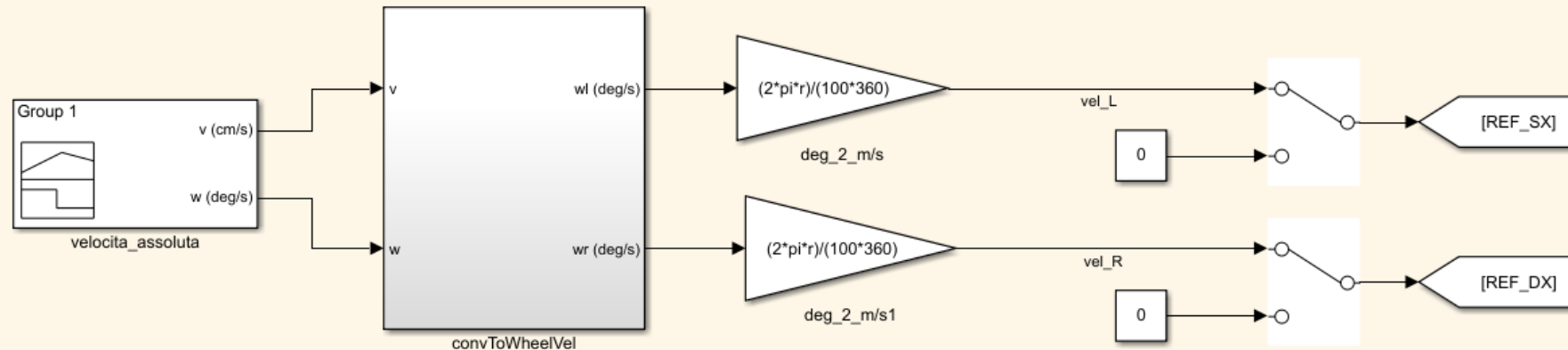




# FIRST STEPS

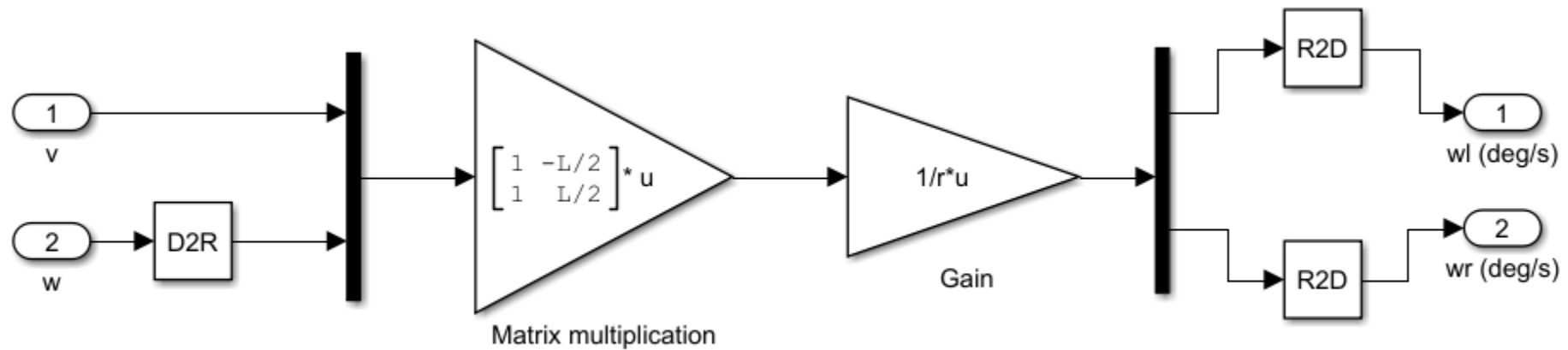
Design of a signal generator for our Duckiebot. The first two blocks are used to generate a simple signal and to convert the velocity into deg/s (this uses the inverse kinematic formula to do so) for the right and left wheel. The gain is only a conversion from deg/s to m/s and the switch is a safety measure

## REF GENERATOR MANUAL



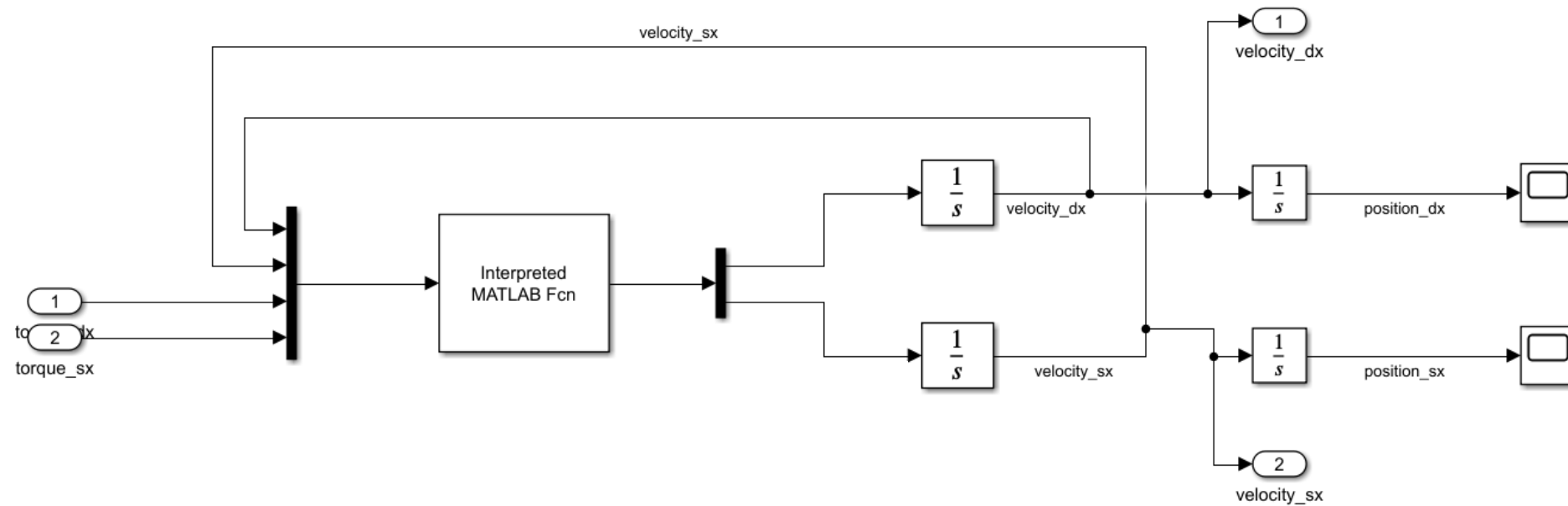
# FIRST STEPS

Matrix and gain, derived from the inverse kinematic formula, are used to calculate the contribution for each wheel



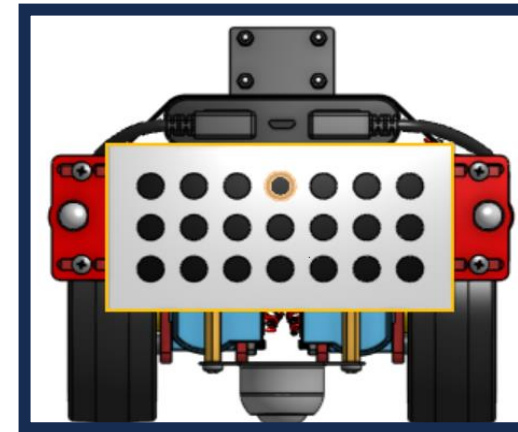
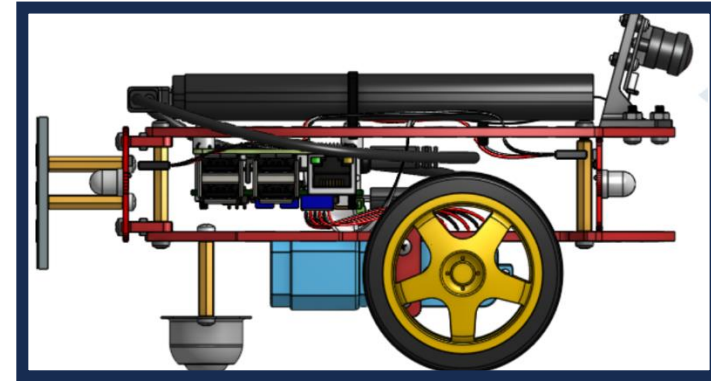
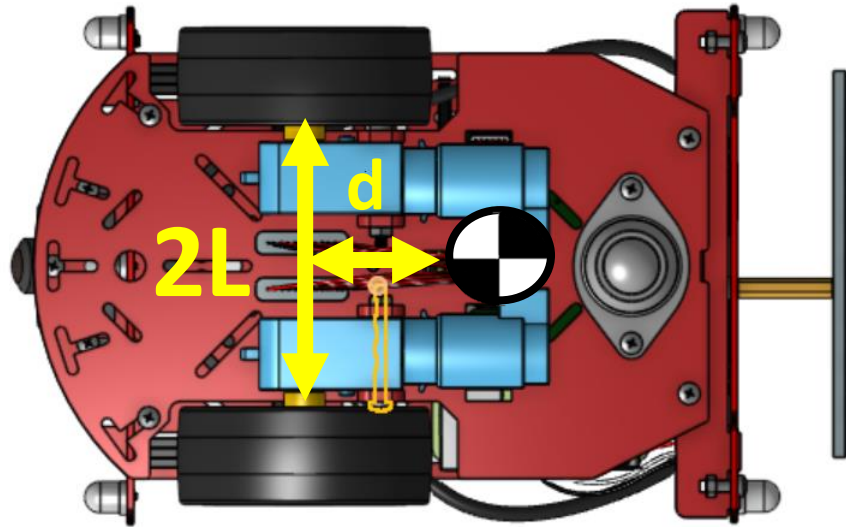
# DYNAMIC MODEL

To tune PID we used the differential mobile robot dynamic model



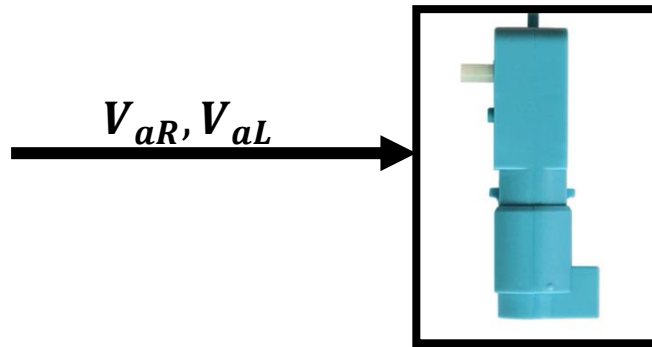
# DYNAMIC MODEL

It's interesting to point out how we found the parameters of the model



# DYNAMIC MODEL

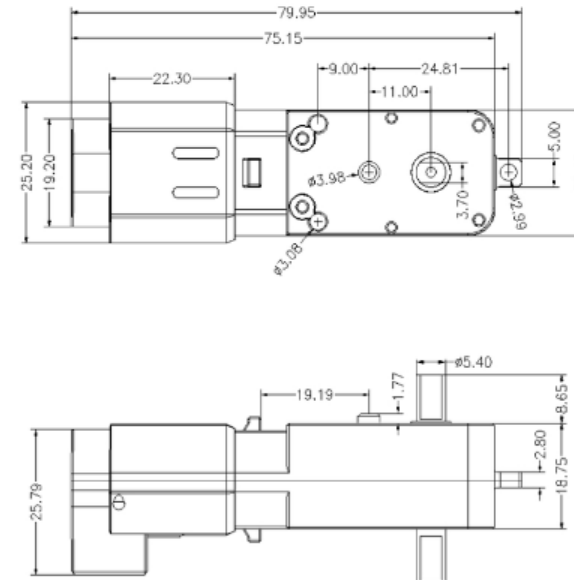
Dynamic model with actuators:



DG01D-E motor with encoder

Specification:  
Dimension: 80x22.4x25.8mm  
Voltage: 3V~9V  
Gear ratio: 1:48  
Speed: 90RPM (4.5V)  
Color: blue color  
Gear material: metal

Dimension drawing:



I'm afraid this information is not all practically available. This is the datasheet available on the motors - depending on which model you are using:

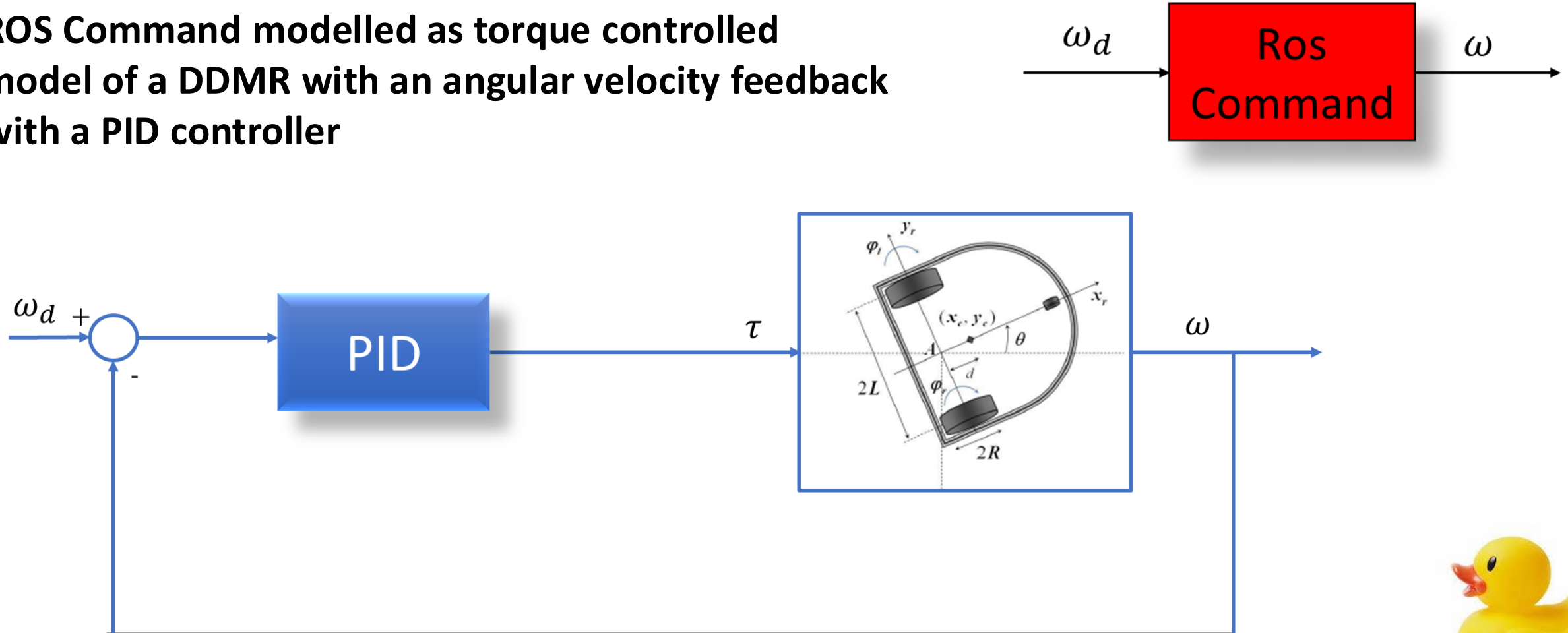
<https://hubs.ly/Q028-NI-0>

Jacopo



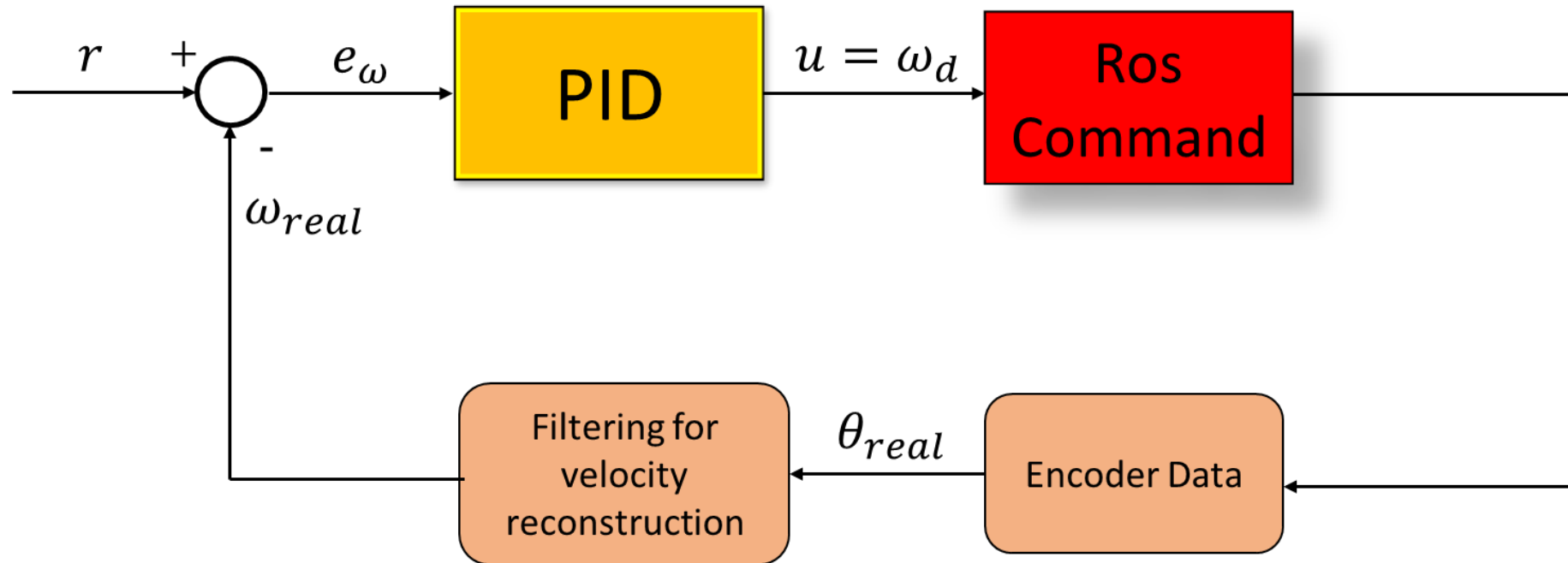
# ROS MODELLING COMMAND

ROS Command modelled as torque controlled model of a DDMR with an angular velocity feedback with a PID controller





# VELOCITY FEEDBACK CONTROL

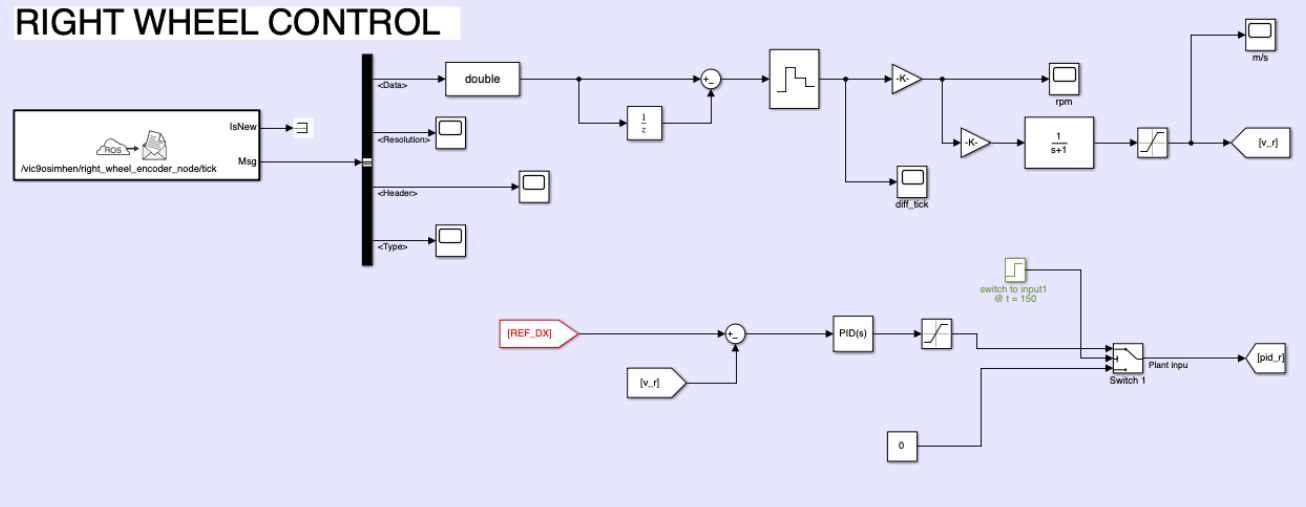


$$PID(s) = 1.3 e_\omega + 1.4 \int e_\omega dt + 0.1 \dot{e}_\omega$$

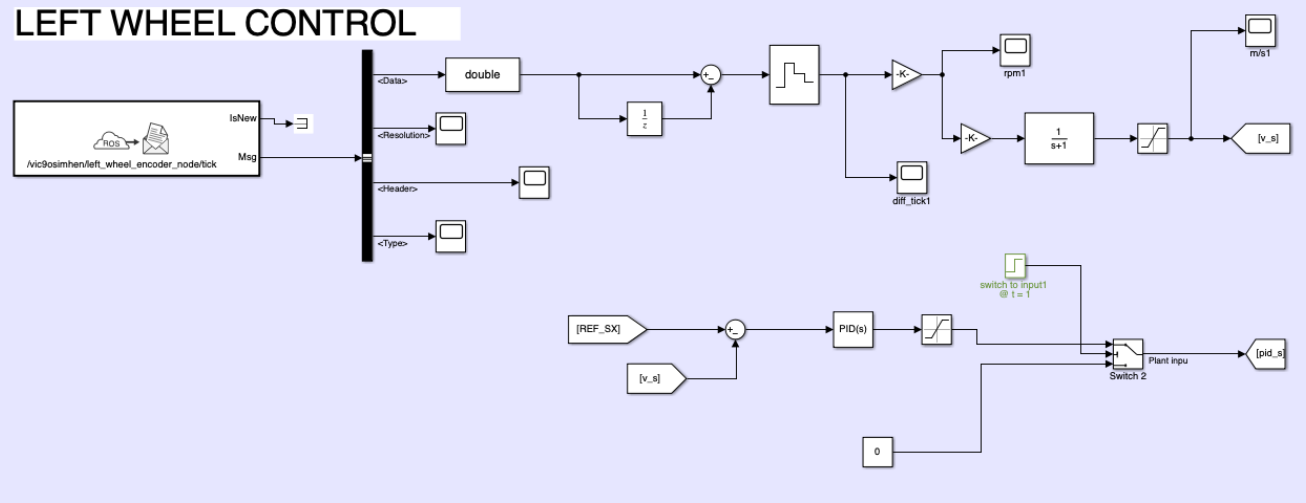


# VELOCITY FEEDBACK CONTROL

## RIGHT WHEEL CONTROL

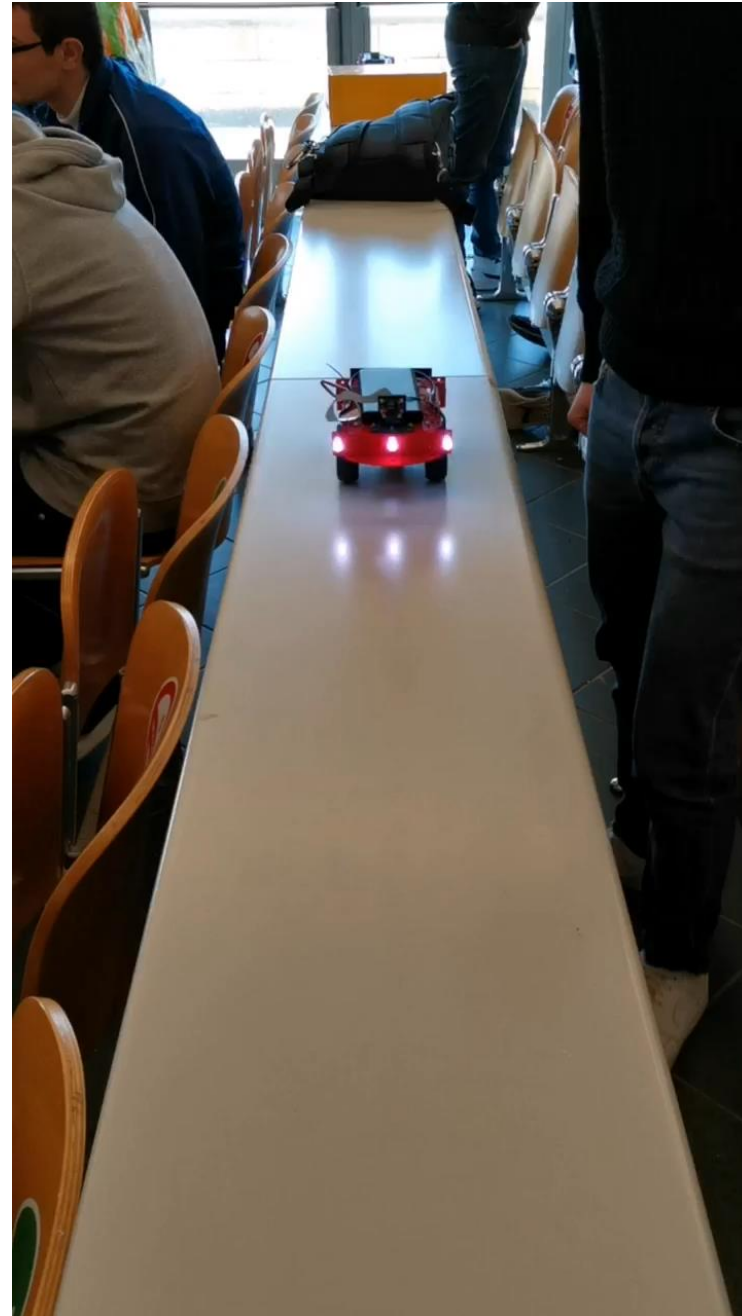


## LEFT WHEEL CONTROL



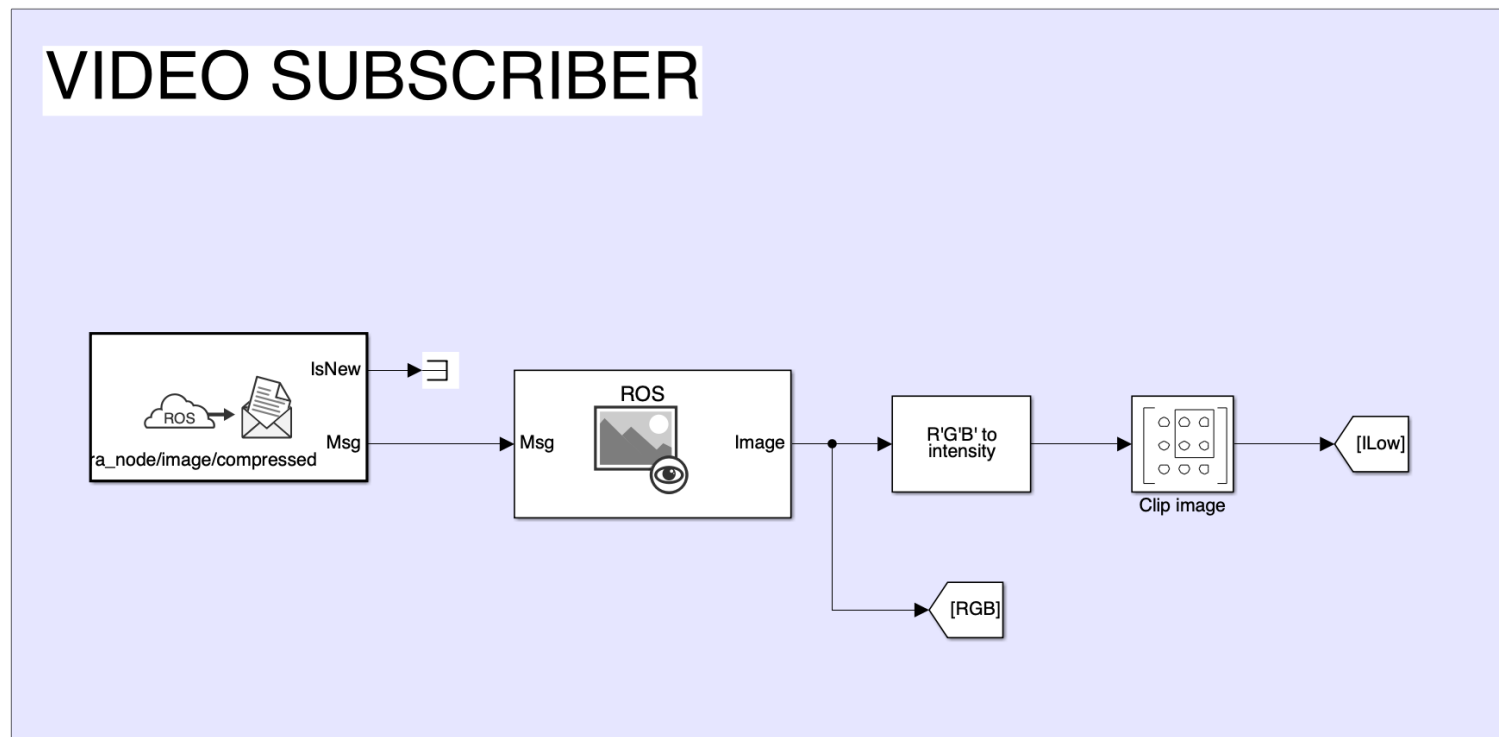
# HANDS - ON

We therefore tested the speed feedback control with a linear velocity reference



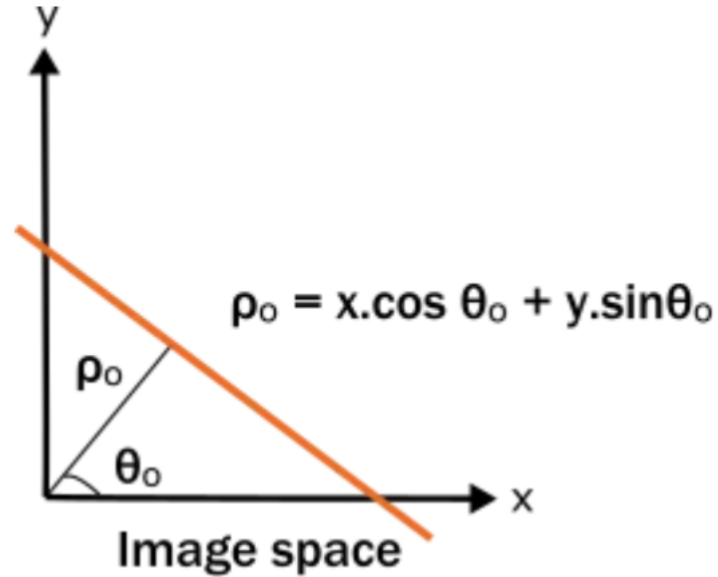
# LINE DETECTION

In autonomous driving the line detection constitutes the foundational concept underlying the lane keeping; it involves identifying lines in a live video sourced from a camera using the Hough transform that converts image points into a parametric space (Hough Space)



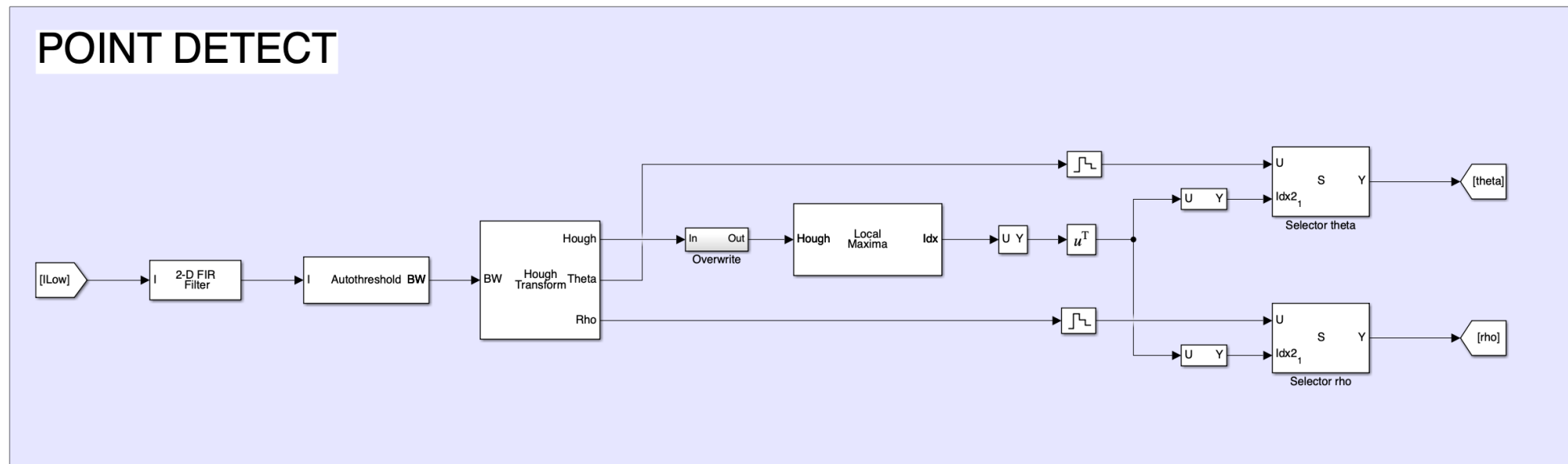
# LINE DETECTION

The Hough transform is a method employed in image processing and computer vision to detect shapes, in particular lines or geometric forms such as circles or straight segments.



# LINE DETECTION

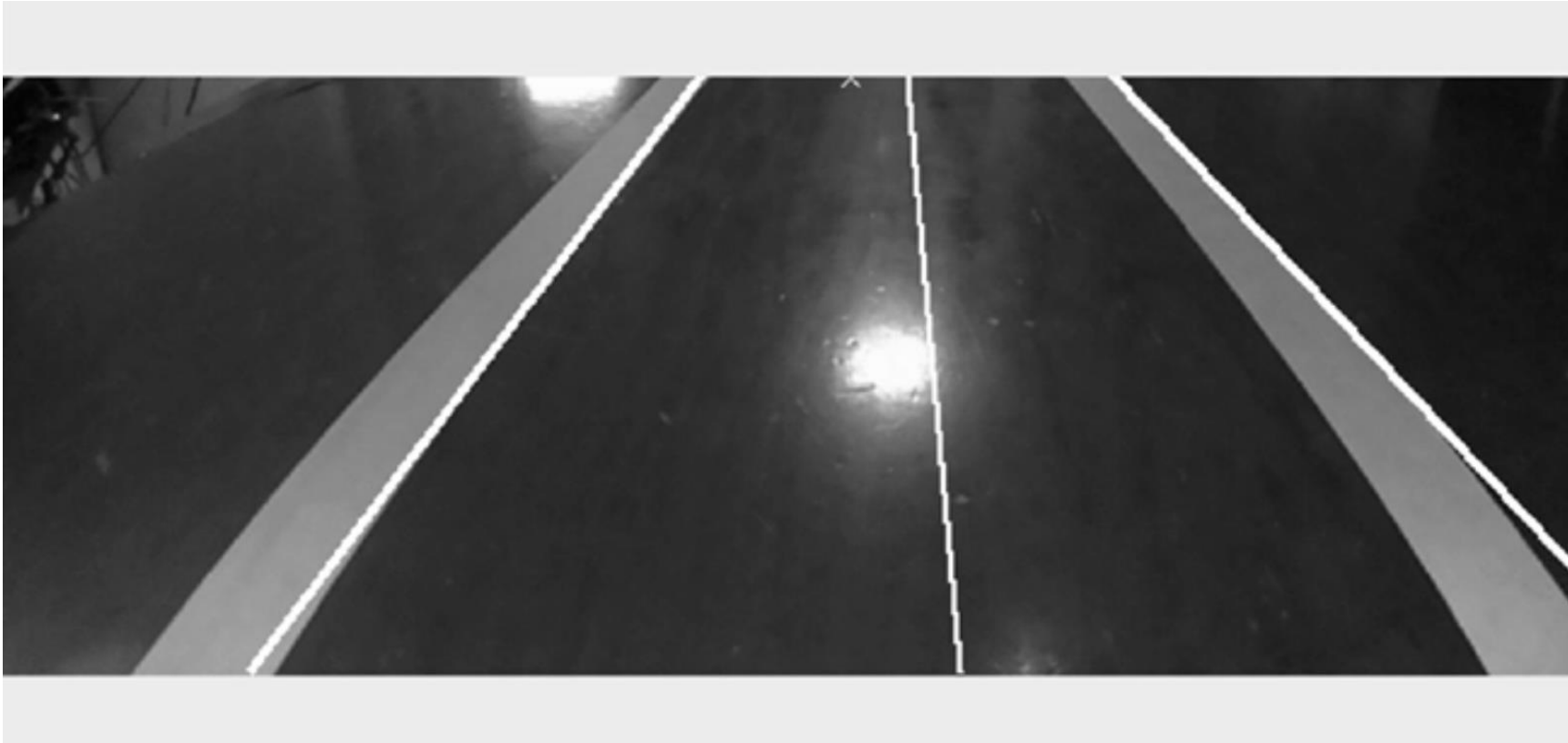
Then, the image undergoes filtering, transformed into black and white and processed by the Hough transform to identify the lane lines. Subsequently we locate the endpoints of the two major lines thanks to the Find Local Maxima block. Utilizing a selector, we extract the indices that enable us to identify the rho and theta values linked to the lane lines.





# LANE KEEPING

**Concept primarily applied in autonomous driving and Advanced Driver Assistance Systems (ADAS), is designed to help vehicles maintain a central position within their lane**



# LANE KEEPING

Line detection

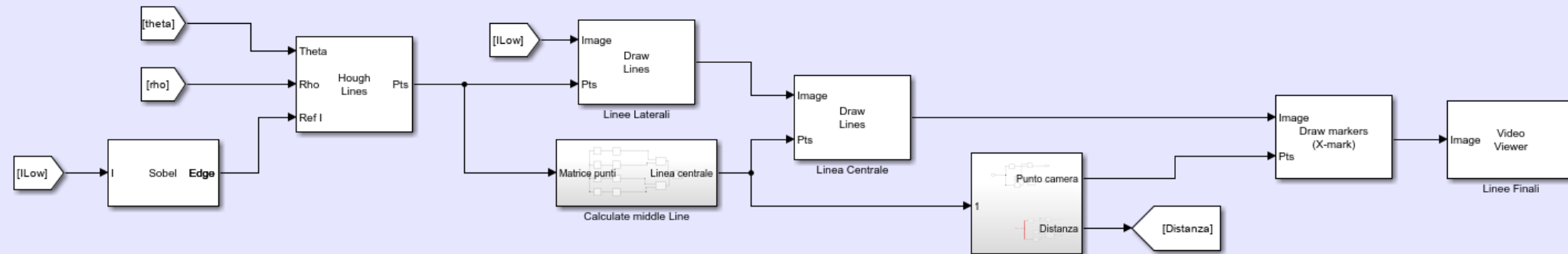


Retrieve the central line

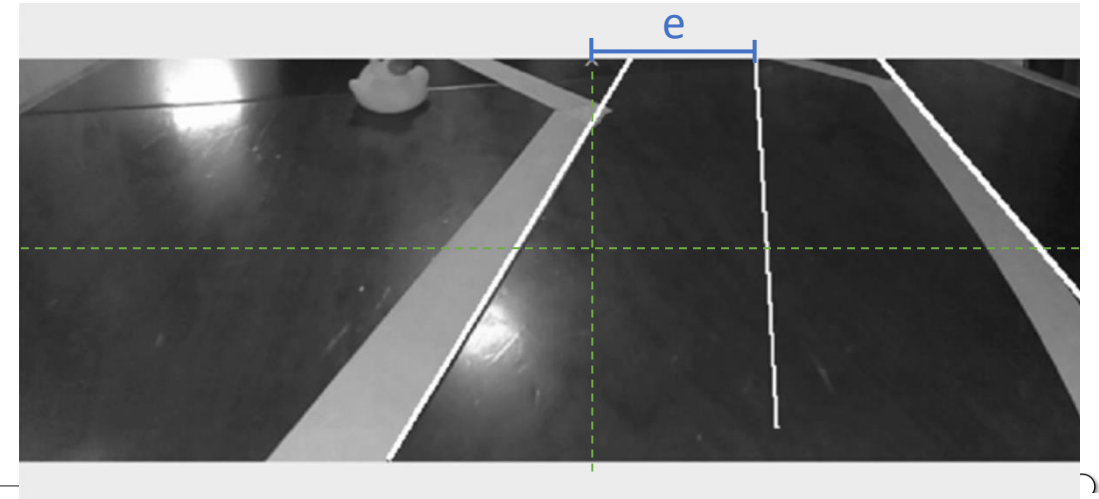
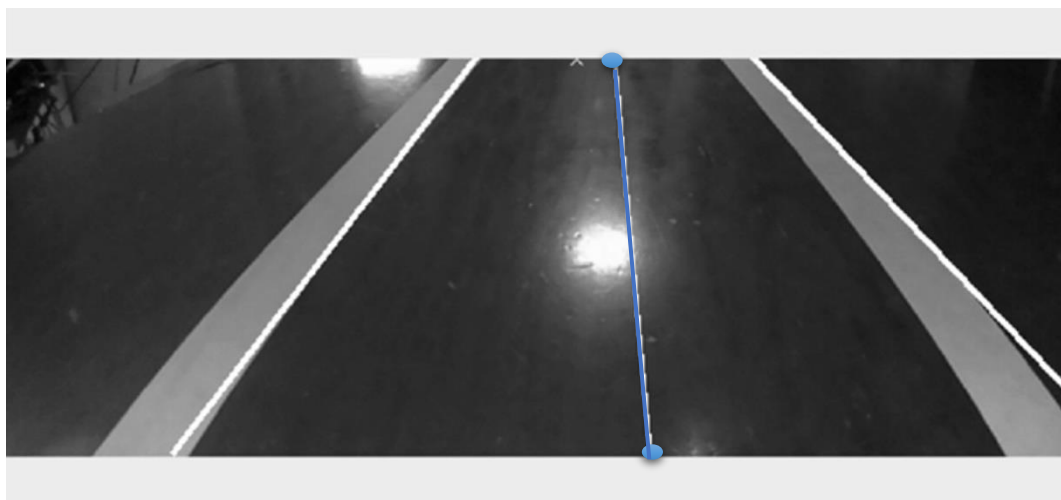
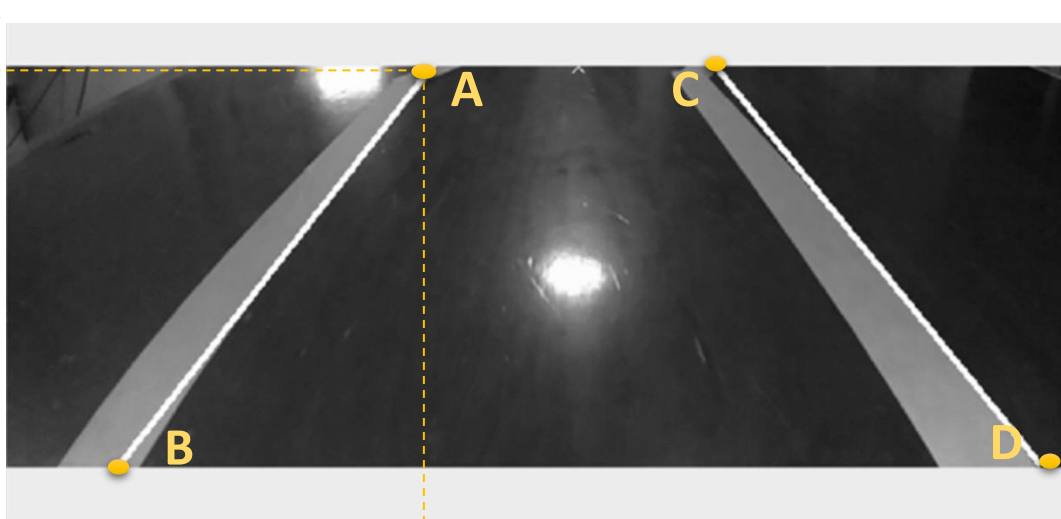


Compute error

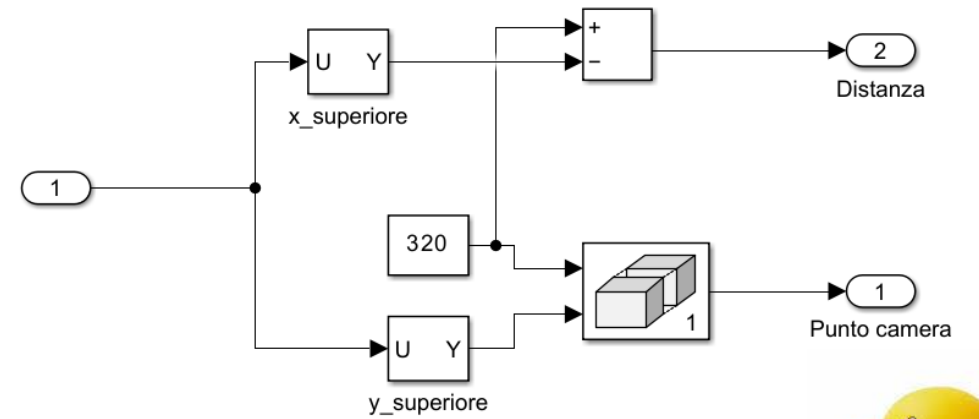
## DRAW LINES



# ERROR COMPUTATION

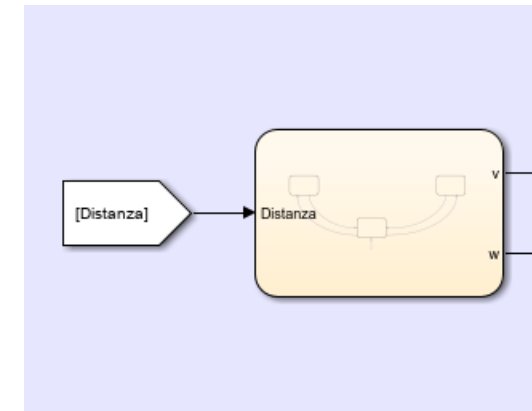
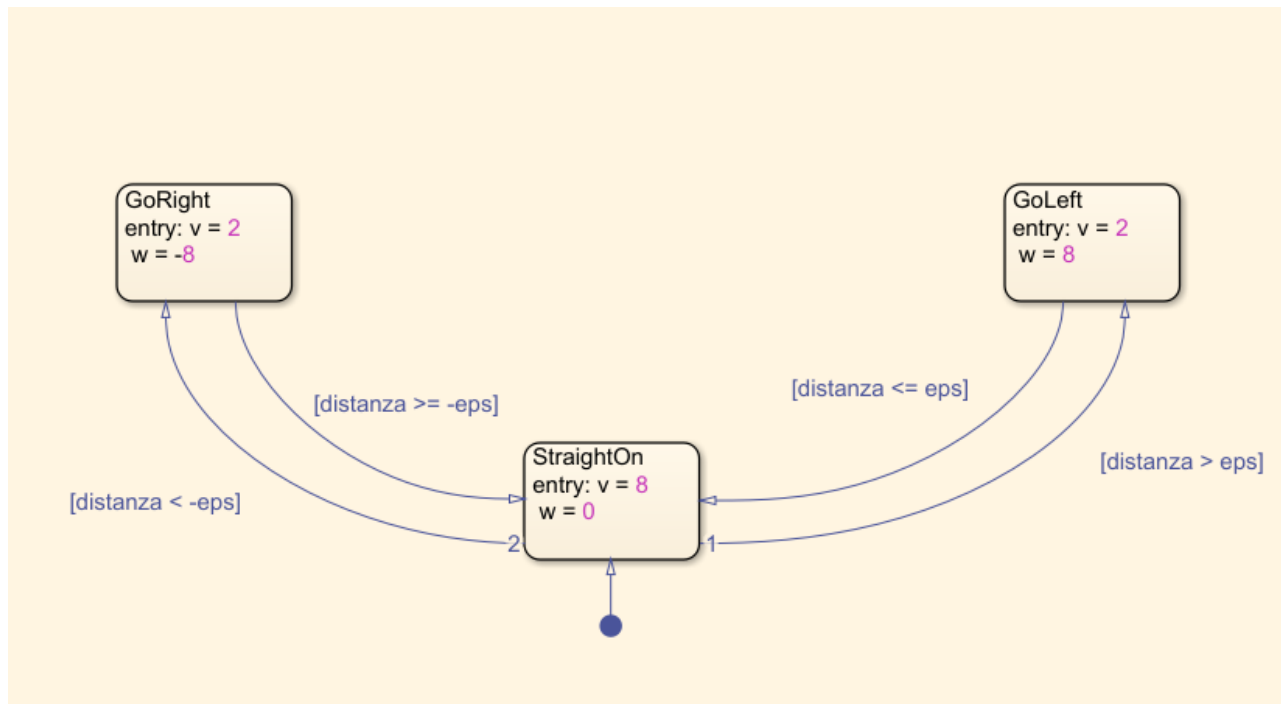


1  
Matrice punti



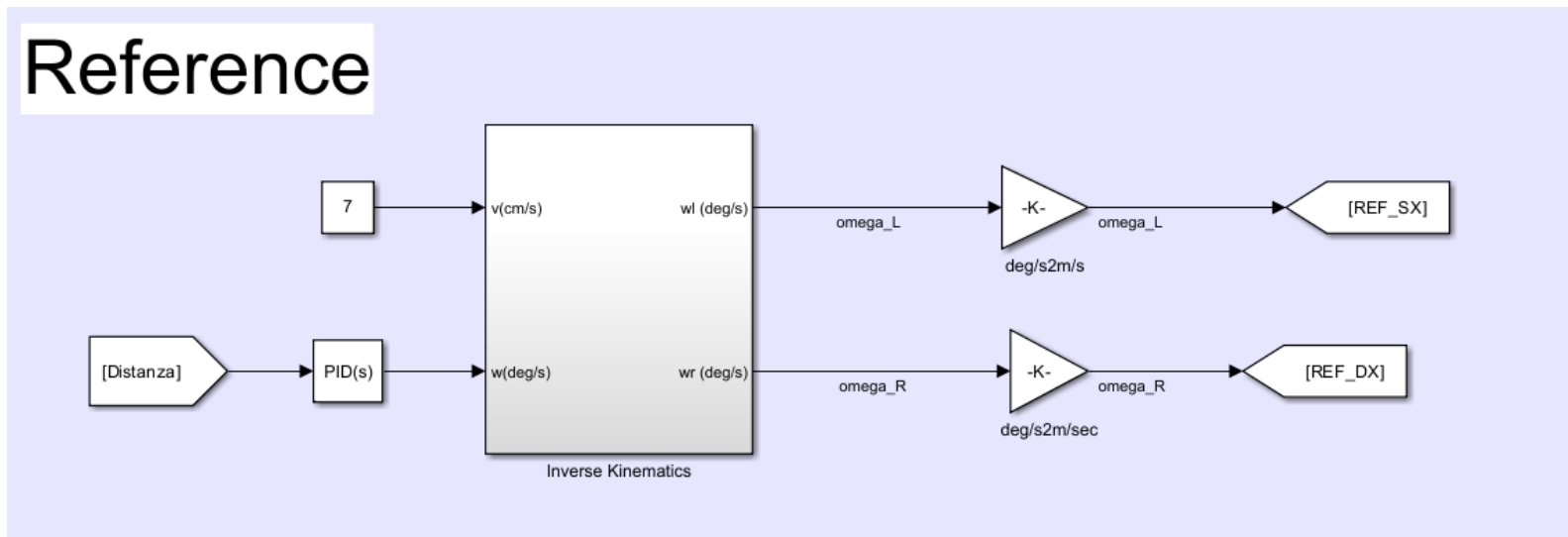
# SWITCHING CONTROLLER

An switching control was implemented using Simulink's 'Chart' block, which is based on graphical programming and allows creating state diagram and finite state machine-based systems



# PID CONTROLLER

For more accurate control, a PID control was later implemented. Based on the error, it generates the rotational speed to command the machine, always maintaining a constant tangential speed

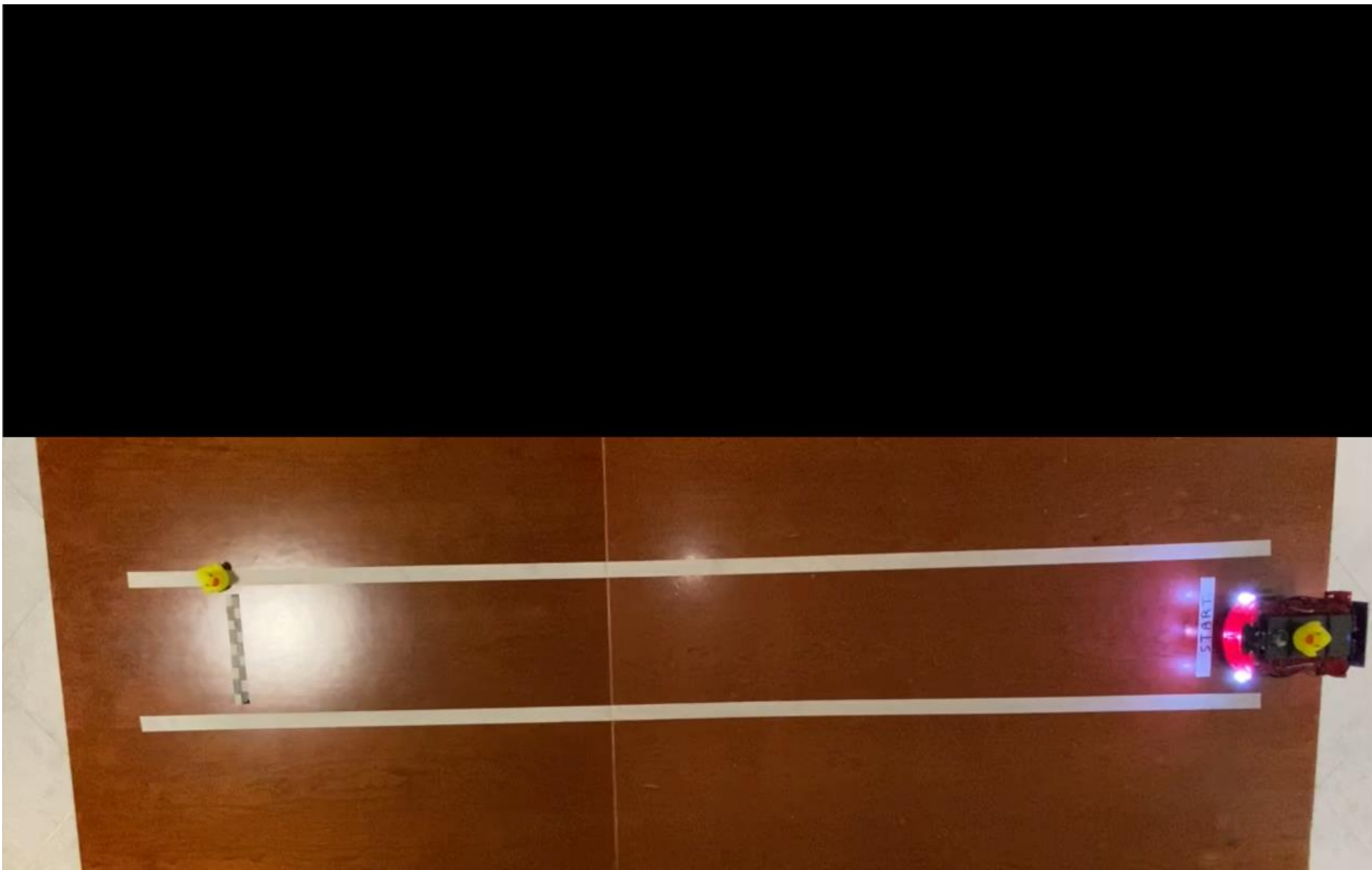


# HANDS - ON

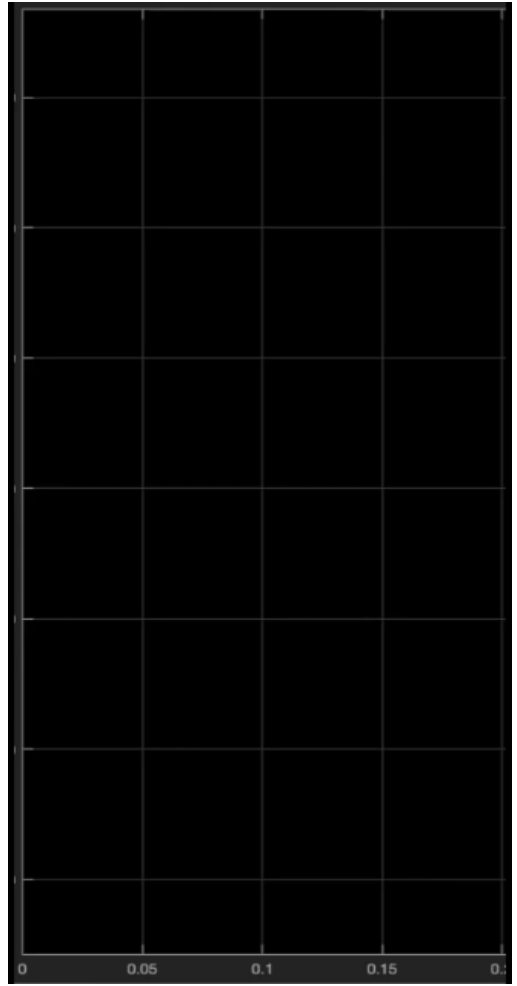
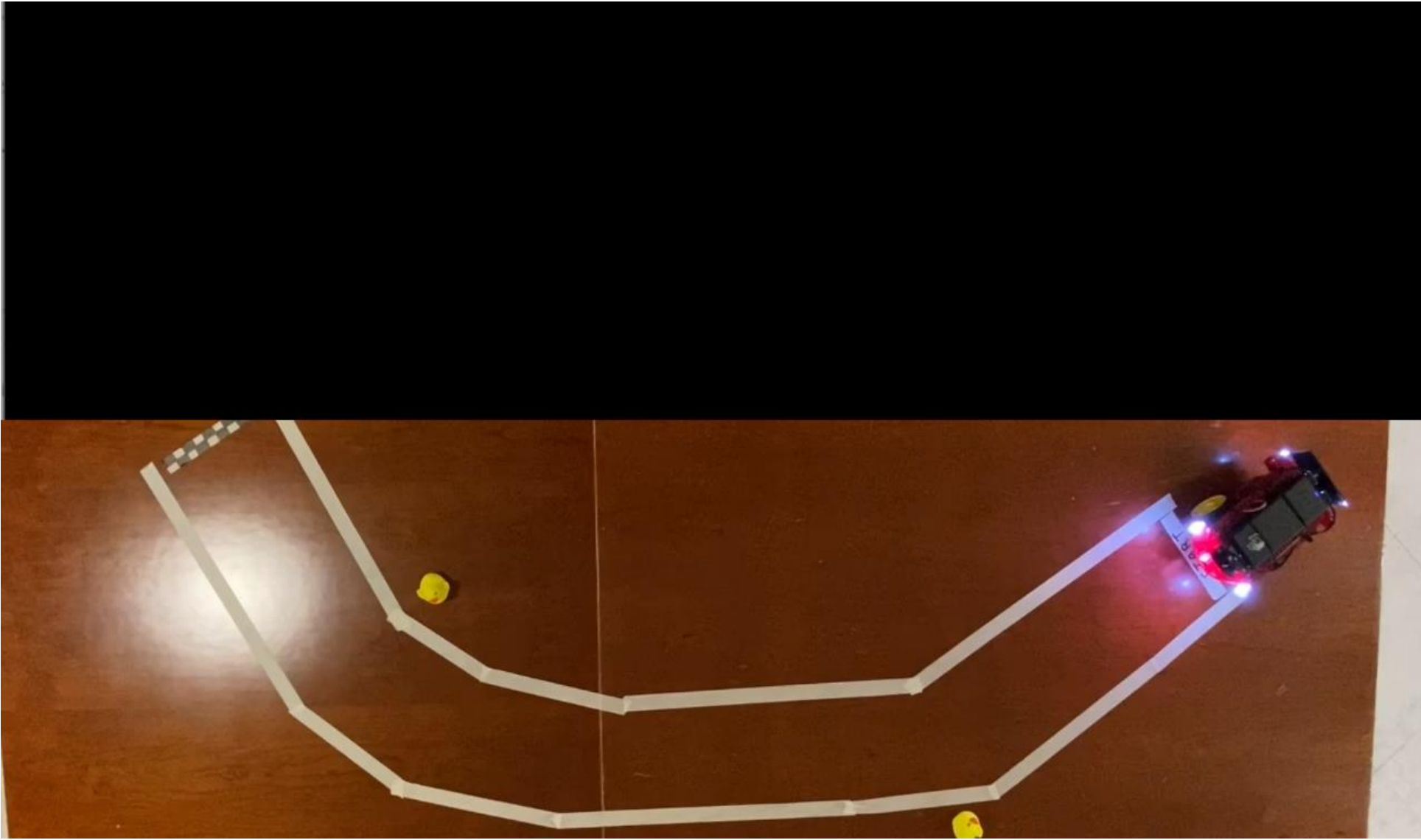
**Once the lane keeping algorithm was developed, we carried out some tests to verify that it worked correctly**

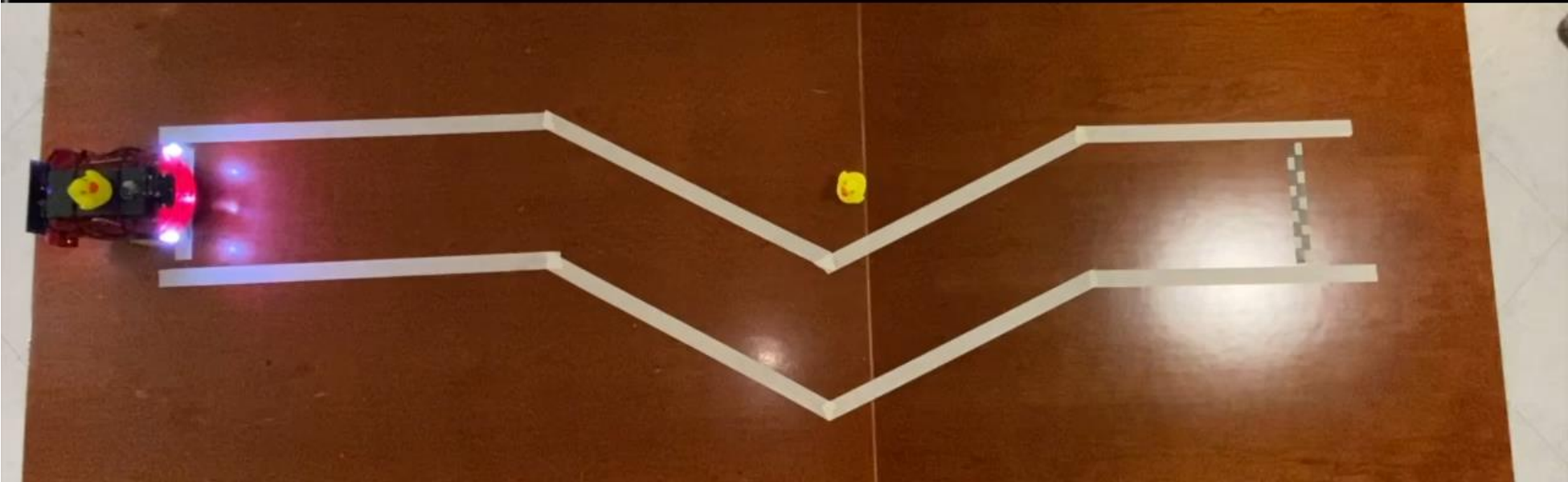
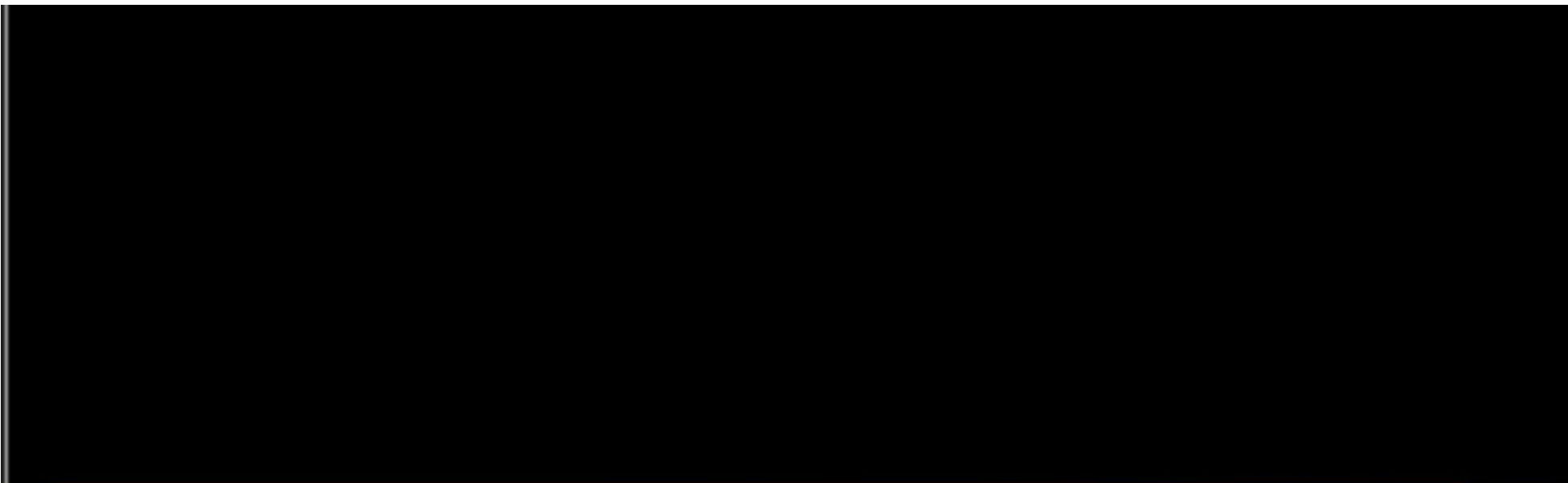
- **Linear trajectory**
- **Curved trajectory**
- **Broken line trajectory**



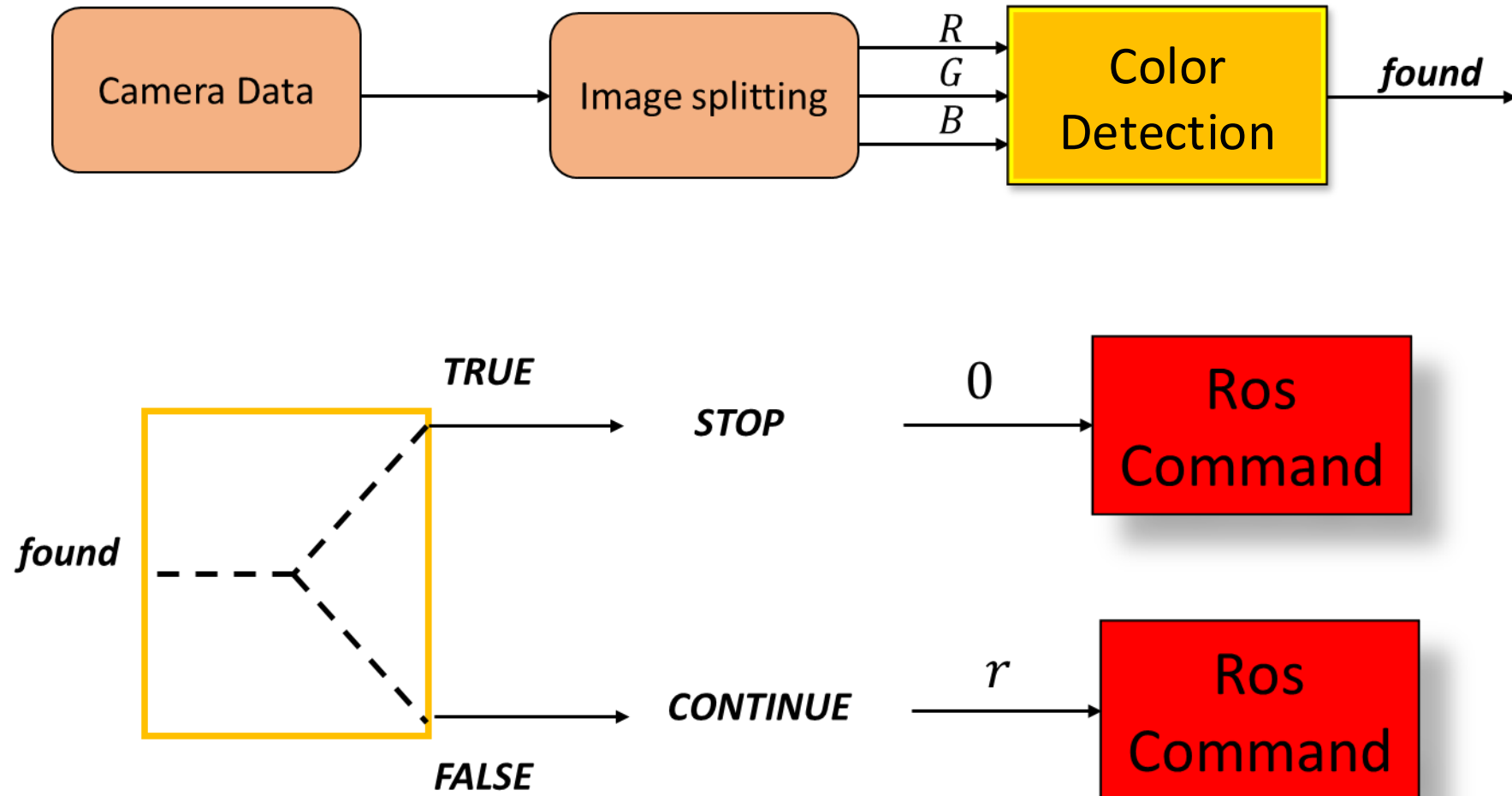






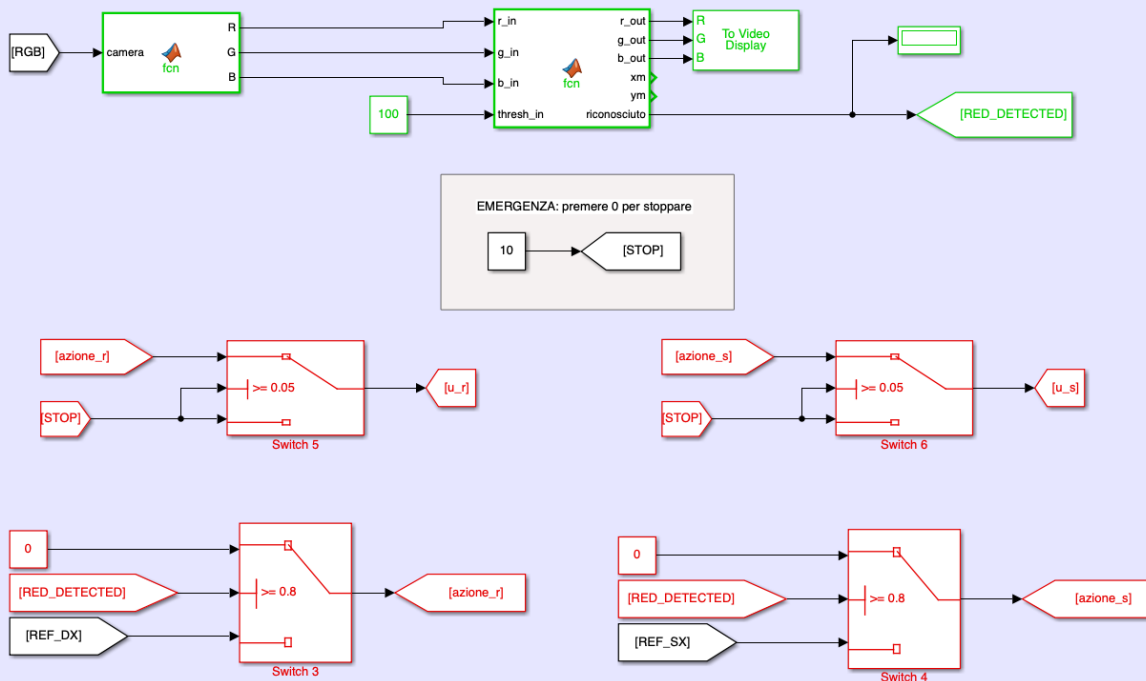


# COLOR DETECTION



# COLOR DETECTION

## COLOR DETECTION



```
function [r_out, g_out, b_out, xm, ym, riconosciuto] = fcn(r_in, g_in, b_in, thresh_in)
    % Get RGB color
    r = r_in;
    g = g_in;
    b = b_in;
    thresh = thresh_in;
    % Compute red channel
    justRed = r - g/2 - b/2;
    % Threshold
    bw = justRed > thresh;
    % Select the right color
    redCondition = (r > 120) & (g < 100) & (b < 100);
    bw = bw & redCondition;
    % Find the mean
    [x, y] = find(bw);
    xm = mean(x);
    ym = mean(y);
    % Compute the marker
    xx = max(1, xm - 5):min(xm + 5, size(bw, 1));
    yy = max(1, ym - 5):min(ym + 5, size(bw, 2));
    bwbw = zeros(size(bw), 'uint8');
    bwbw(uint16(xx), uint16(yy)) = 255;
    % Write the marker on the original frame
    r_out = uint8(double(r_in) - double(bwbw));
    g_out = uint8(double(g_in) + double(bwbw));
    b_out = uint8(double(b_in) - double(bwbw));
    % Check if the color is detected
    if any(bw(:))
        riconosciuto = 1;
    else
        riconosciuto = 0;
    end
end
```

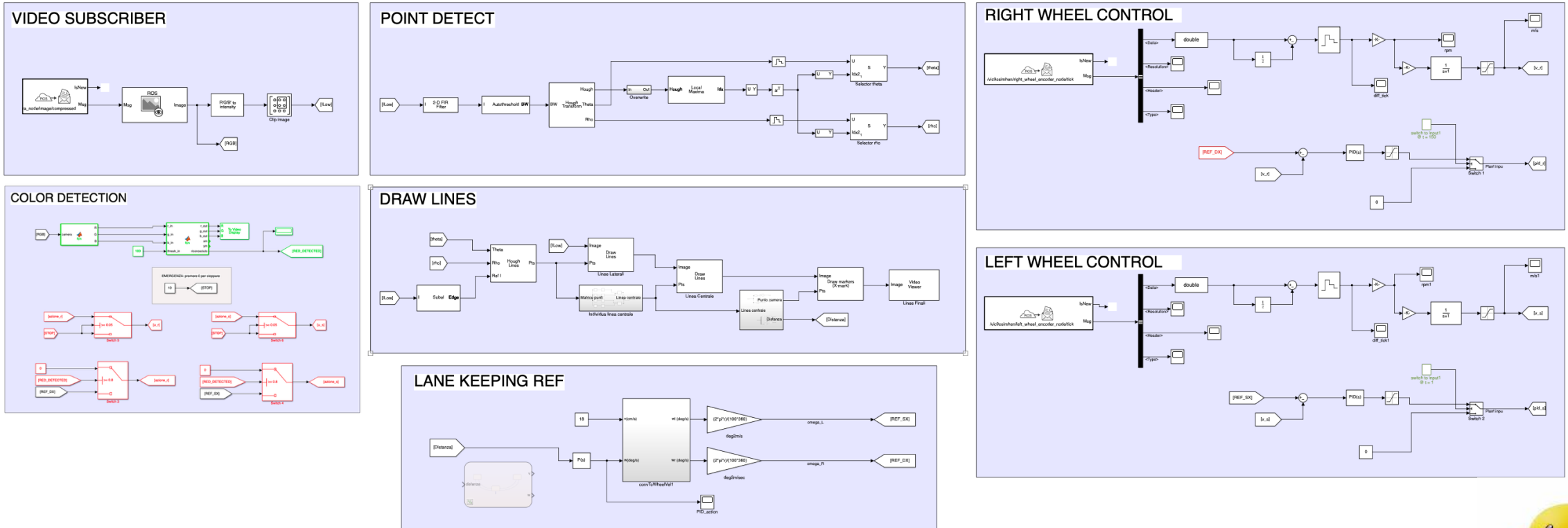


# COLOR DETECTION



# CONTROL MERGE

After developing the cruise feedback control and lane keeping, we tried to combine the two controls in order to achieve more precise and punctual lane keeping





# BUILD & RUN

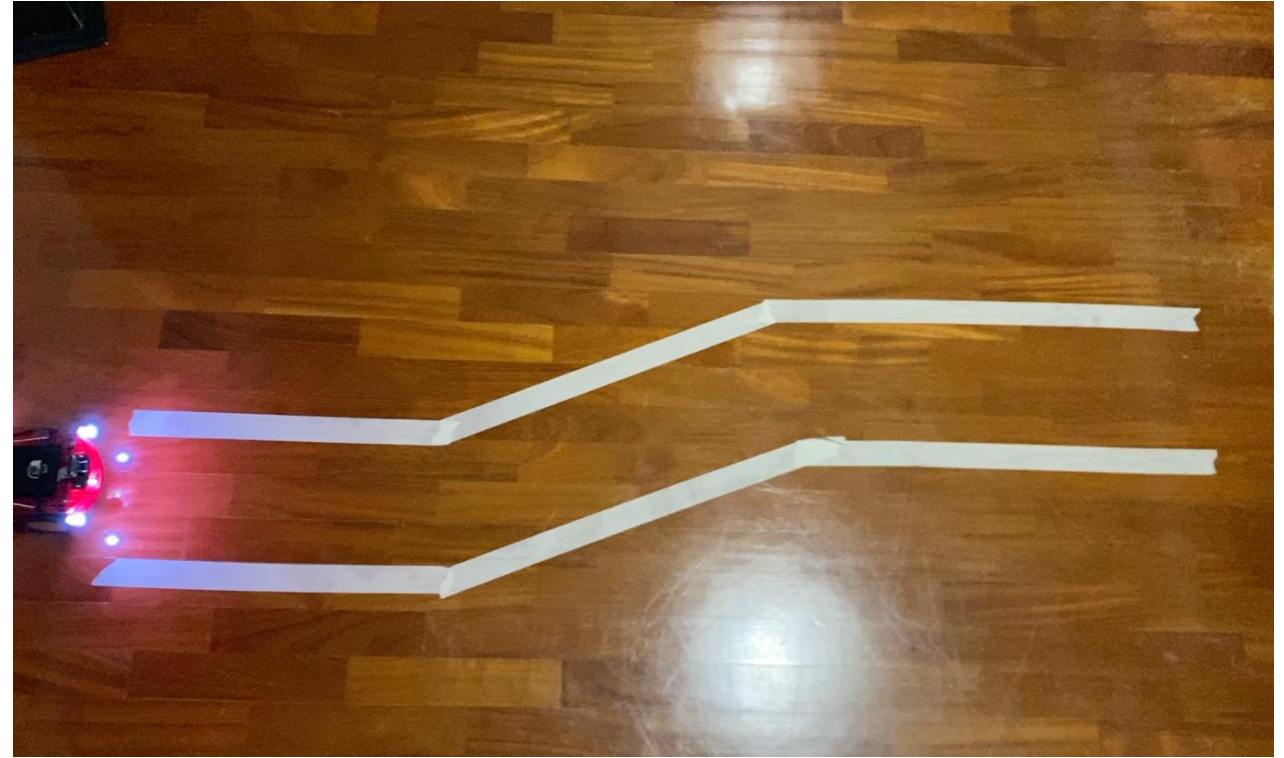
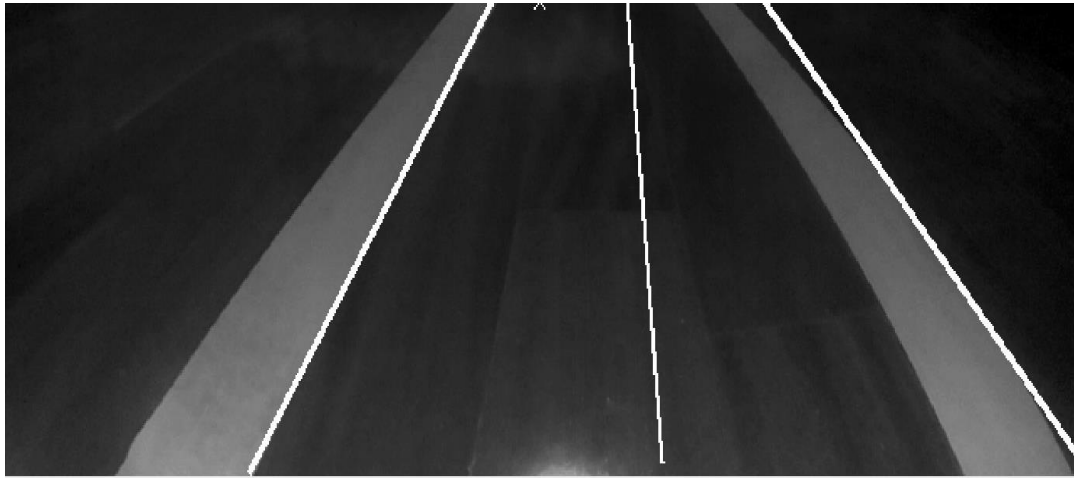
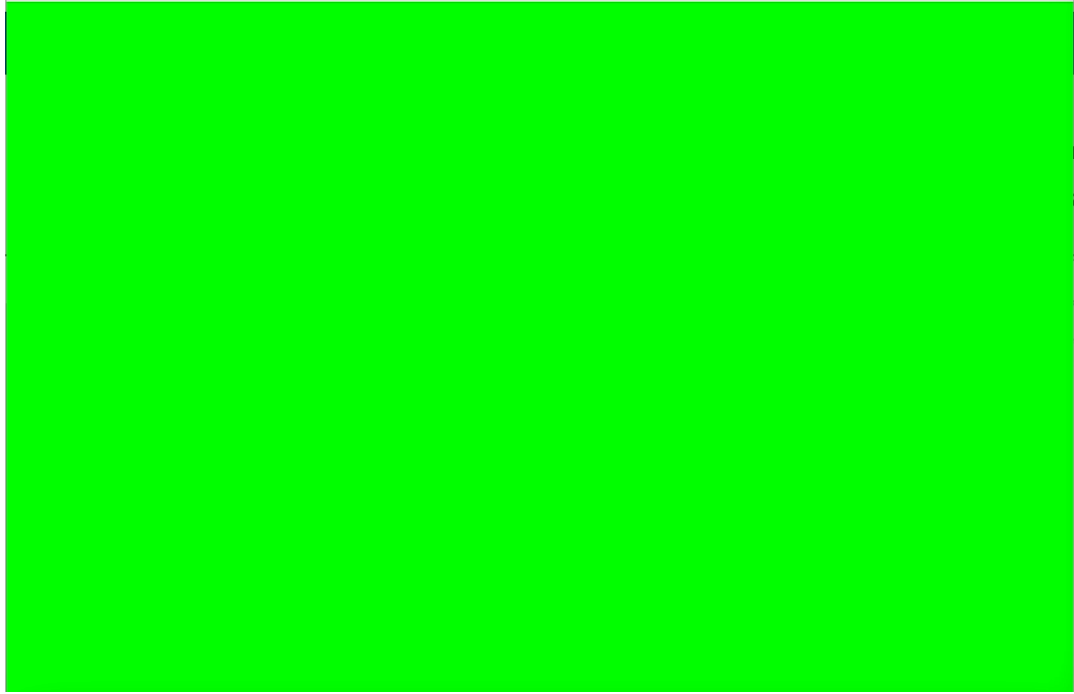
However, we found that the lane-keeping pattern significantly slows down Simulink's simulation time due to the camera's ROS subscriber. This causes a lot of latency that prevents the speed control scheme from working properly

We hypothesized that by loading the Simulink model directly onto the DuckieBot, latency issues could be resolved. However, we encountered an additional error when trying to build the code via the ROS toolbox due to the ROS architecture on which the DuckieBot is based, which uses Docker

For this reason, we decide to merge the lane keeping control and the color detection algorithm







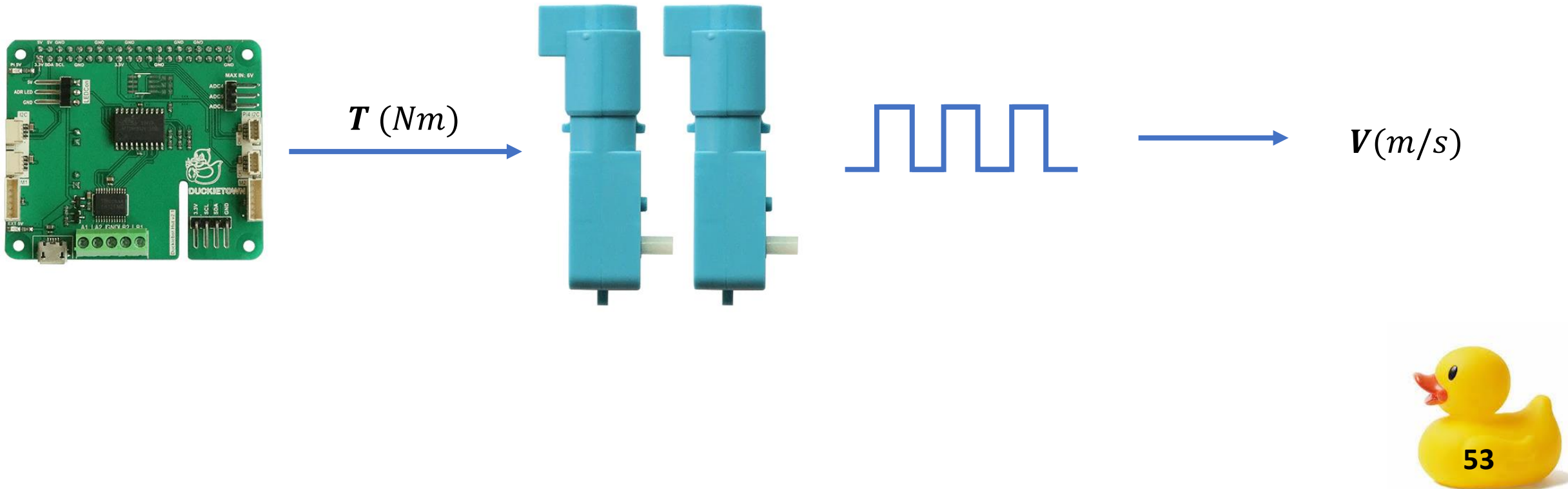
# FUTURE IMPLEMENTATIONS

- Solve latency problems



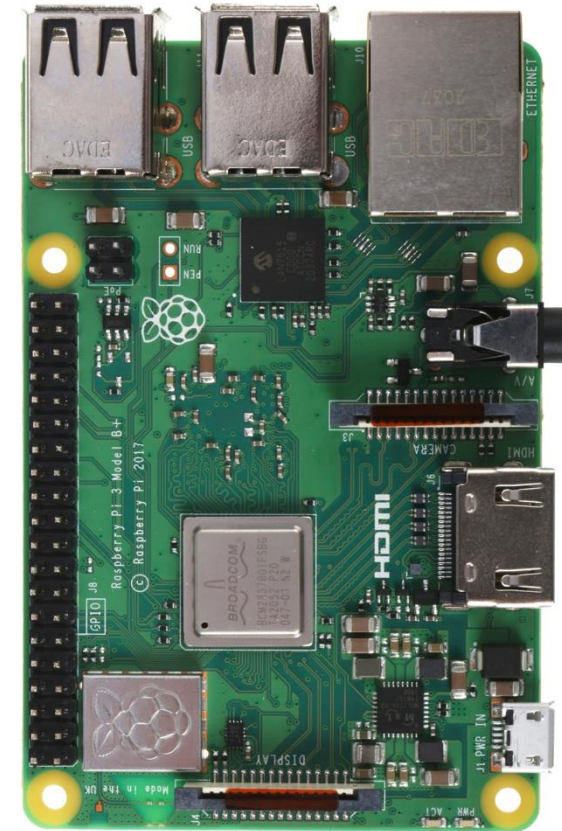
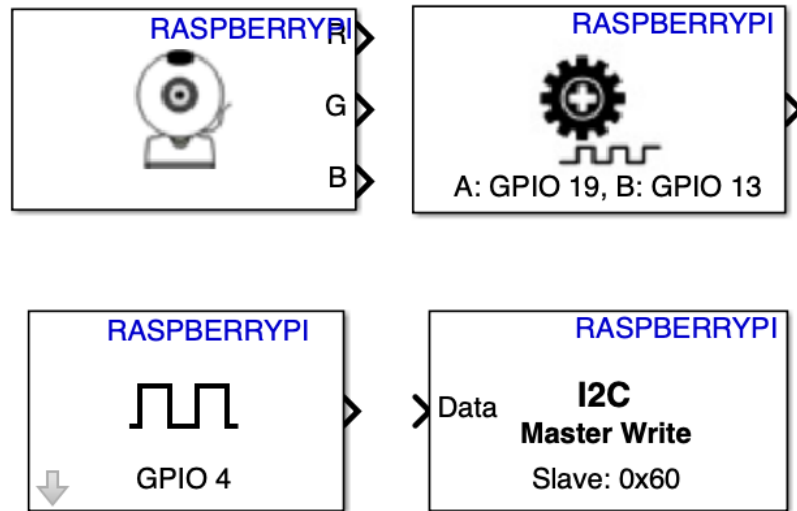
# FUTURE IMPLEMENTATIONS

- Map the HAT to command the motors in torque and retrieve the velocity in a proper way



# FUTURE IMPLEMENTATIONS

- Building the control with Raspberry Toolbox







THANKS FOR YOUR ATTENTION